

UMS - Linux Kernel Module

Generated by Doxygen 1.8.17

1 AOSV Final Project Sources	1
2 Bug List	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	7
4.1 File List	7
5 Data Structure Documentation	9
5.1 hlist_head Struct Reference	9
5.2 hlist_node Struct Reference	10
5.3 list_head Struct Reference	10
5.4 procfs_proc_umdata Struct Reference	11
5.4.1 Detailed Description	11
5.5 procfs_sched_umdata Struct Reference	11
5.5.1 Detailed Description	11
5.6 procfs_worker_umdata Struct Reference	12
5.6.1 Detailed Description	12
5.7 ums_proc Struct Reference	12
5.7.1 Detailed Description	13
5.8 ums_sched Struct Reference	13
5.8.1 Detailed Description	14
5.8.2 Field Documentation	14
5.8.2.1 current_worker	14
5.9 ums_sched_arg Struct Reference	15
5.9.1 Field Documentation	15
5.9.1.1 cpuid	15
5.10 ums_thread Struct Reference	16
5.10.1 Detailed Description	16
5.11 ums_usr_worker Struct Reference	16
5.11.1 Detailed Description	16
5.11.2 Field Documentation	16
5.11.2.1 cpuid	17
5.11.2.2 tid	17
5.12 ums_worker Struct Reference	17
5.12.1 Detailed Description	18
5.12.2 Field Documentation	18
5.12.2.1 state	18
5.13 ums_worker_arg Struct Reference	19
5.13.1 Detailed Description	19
5.14 ums_worker_node Struct Reference	20
5.14.1 Detailed Description	20

6 File Documentation	21
6.1 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ioctl.c File Reference	21
6.1.1 Detailed Description	22
6.1.2 Function Documentation	22
6.1.2.1 ums_ioctl()	22
6.2 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ioctl.h File Reference	22
6.2.1 Detailed Description	23
6.2.2 Macro Definition Documentation	24
6.2.2.1 DEQUEUE_LIST	24
6.2.2.2 EXECUTE_THREAD	24
6.2.2.3 INIT_WORKER	24
6.2.2.4 REGISTER_SCHED	24
6.2.2.5 REGISTER_WORKER	25
6.2.3 Function Documentation	25
6.2.3.1 ums_ioctl()	25
6.3 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/procfs.c File Reference	25
6.3.1 Detailed Description	26
6.3.2 Function Documentation	27
6.3.2.1 sched_proc_read()	27
6.3.2.2 worker_proc_read()	27
6.4 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/procfs.h File Reference	27
6.4.1 Detailed Description	28
6.4.2 Function Documentation	28
6.4.2.1 sched_proc_read()	29
6.4.2.2 worker_proc_read()	29
6.5 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/shared.h File Reference	29
6.5.1 Detailed Description	31
6.5.2 Variable Documentation	31
6.5.2.1 ncpus	31
6.6 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/shared.h File Reference	32
6.6.1 Detailed Description	34
6.6.2 Variable Documentation	34
6.6.2.1 ncpus	34
6.7 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ums.c File Reference	35
6.7.1 Detailed Description	35
6.8 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.c File Reference	35
6.8.1 Detailed Description	36
6.8.2 Function Documentation	37
6.8.2.1 DequeueUmsCompletionListItems()	37
6.8.2.2 EnterUmsSchedulingMode()	37
6.8.2.3 ExecuteUmsThread()	38
6.8.2.4 ums_destroy()	38

6.8.2.5 ums_init()	38
6.8.2.6 ums_worker_create()	38
6.8.2.7 ums_worker_list_insert()	39
6.9 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/utils.c File Reference	39
6.9.1 Detailed Description	41
6.9.2 Function Documentation	41
6.9.2.1 cleanup_process()	41
6.9.2.2 dequeue_list()	41
6.9.2.3 execute_thread()	42
6.9.2.4 find_ums_proc()	42
6.9.2.5 init_worker_node()	42
6.9.2.6 register_ums_process()	43
6.9.2.7 register_ums_scheduler()	43
6.9.2.8 register_usr_worker()	43
6.9.2.9 terminate_ums_process()	44
6.10 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/utils.c File Reference	44
6.10.1 Detailed Description	45
6.10.2 Function Documentation	45
6.10.2.1 dup_worker_list()	45
6.10.2.2 find_next_zero_bit()	46
6.10.2.3 find_worker_tid()	46
6.10.2.4 open_device()	46
6.10.2.5 sched_wrap_routine()	47
6.10.2.6 worker_wrap_routine()	47
6.11 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/utils.h File Reference	47
6.11.1 Detailed Description	49
6.11.2 Function Documentation	49
6.11.2.1 cleanup_process()	49
6.11.2.2 dequeue_list()	49
6.11.2.3 execute_thread()	50
6.11.2.4 find_ums_proc()	50
6.11.2.5 init_worker_node()	50
6.11.2.6 register_ums_process()	51
6.11.2.7 register_ums_scheduler()	51
6.11.2.8 register_usr_worker()	52
6.11.2.9 terminate_ums_process()	52
6.12 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/utils.h File Reference	52
6.12.1 Detailed Description	54
6.12.2 Function Documentation	54
6.12.2.1 dup_worker_list()	54
6.12.2.2 find_next_zero_bit()	54
6.12.2.3 find_worker_tid()	55

6.12.2.4	open_device()	55
6.12.2.5	sched_wrap_routine()	55
6.12.2.6	worker_wrap_routine()	56
6.13	/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/bitmap.h File Reference	56
6.13.1	Detailed Description	58
6.13.2	Macro Definition Documentation	58
6.13.2.1	DECLARE_BITMAP	58
6.13.2.2	GET_BIT	58
6.13.2.3	SET_BIT	59
6.13.2.4	UNSET_BIT	59
6.14	/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/list.h File Reference	59
6.14.1	Detailed Description	61
6.14.2	Macro Definition Documentation	61
6.14.2.1	__list_for_each	61
6.14.2.2	container_of	62
6.14.2.3	hlist_for_each_entry	62
6.14.2.4	hlist_for_each_entry_continue	63
6.14.2.5	hlist_for_each_entry_from	63
6.14.2.6	hlist_for_each_entry_safe	63
6.14.2.7	hlist_for_each_safe	64
6.14.2.8	list_entry	64
6.14.2.9	list_first_entry	65
6.14.2.10	list_for_each	65
6.14.2.11	list_for_each_entry	65
6.14.2.12	list_for_each_entry_continue	66
6.14.2.13	list_for_each_entry_continue_reverse	66
6.14.2.14	list_for_each_entry_from	66
6.14.2.15	list_for_each_entry_reverse	67
6.14.2.16	list_for_each_entry_safe	67
6.14.2.17	list_for_each_entry_safe_continue	68
6.14.2.18	list_for_each_entry_safe_from	68
6.14.2.19	list_for_each_entry_safe_reverse	69
6.14.2.20	list_for_each_prev	69
6.14.2.21	list_for_each_prev_safe	69
6.14.2.22	list_for_each_safe	70
6.14.2.23	list_prepare_entry	70
6.14.2.24	list_safe_reset_next	71
6.15	/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h File Reference	71
6.15.1	Detailed Description	72
6.15.2	Function Documentation	73
6.15.2.1	DequeueUmsCompletionListItems()	73
6.15.2.2	EnterUmsSchedulingMode()	73

6.15.2.3 ExecuteUmsThread()	74
6.15.2.4 ums_destroy()	74
6.15.2.5 ums_init()	74
6.15.2.6 ums_worker_create()	74
6.15.2.7 ums_worker_list_insert()	75
Index	77

Chapter 1

AOSV Final Project Sources

A.Y. 2020/2021

Author: Nalin Dhingra (1967105)

The sources are structured in this way:

- `lib/` contains the files related to the userspace library
- `dev/` contains the files related to the kernelspace module
- `usr/` contains the applications that from userspace interact with the kernel module

Chapter 2

Bug List

File [bitmap.h](#)

No known bugs.

File [ioctl.c](#)

No known bugs.

File [ioctl.h](#)

No known bugs.

File [list.h](#)

No known bugs.

File [procfs.c](#)

No known bugs.

File [procfs.h](#)

No known bugs.

File [shared.h](#)

No known bugs.

No known bugs.

File [shared.h](#)

No known bugs.

File [ums.c](#)

No known bugs.

File [ums.c](#)

No known bugs.

File [ums.h](#)

No known bugs.

File [utils.c](#)

No known bugs.

File [utils.c](#)

No known bugs.

File [utils.h](#)

No known bugs.

File [utils.h](#)

No known bugs.

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

hlist_head	9
hlist_node	10
list_head	10
procfs_proc_umsdata Structure defining the data needed by a process to manage its procfs instance	11
procfs_sched_umsdata Structure defining the procfs data of a scheduler	11
procfs_worker_umsdata Structure defining the procfs data of a worker	12
ums_proc Structure defining the components that identify a process that is using the UMS driver	12
ums_sched Structure defining a scheduler thread in UMS mode	13
ums_sched_arg	15
ums_thread Structure defining a UMS specific thread	16
ums_usr_worker Structure defining a tuple used to hold registration information about a worker	16
ums_worker Structure defining a worker thread	17
ums_worker_arg Structure used to wrap the arguments of pthread_create within ums_worker_create	19
ums_worker_node Structure used to hold a shared worker in a hashtable	20

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

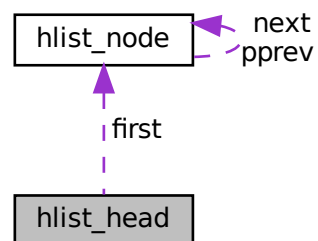
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ ioctl.c	
IOCTL API to interact with the UMS device	21
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ ioctl.h	
Header file for the IOCTL API	22
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ procfs.c	
Implementation of the ProcFS interface related to the UMS driver	25
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ procfs.h	
Header file for the ProcFS interface of the UMS driver	27
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ shared.h	
File used to define global variables declared in shared.h	29
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ ums.c	
Initialization of LKM	35
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ utils.c	
Helper functions and data structures used by LKM	39
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ utils.h	
Headers for helper functions and data structures of LKM	47
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ bitmap.h	
Macros to implement bitmaps	56
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ list.h	
Implementation for linked lists like in the Linux Kernel	59
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ shared.h	
Contains data shared among most of the files of the UMS library	32
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ ums.c	
Implements the main user API to use the functionalities of the UMS driver	35
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ ums.h	
Header file exposing the main user API to interact with the UMS driver	71
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ utils.c	
Contains the helper functions for the UMS library	44
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ utils.h	
All helper functions for ums library	52

Chapter 5

Data Structure Documentation

5.1 hlist_head Struct Reference

Collaboration diagram for hlist_head:



Data Fields

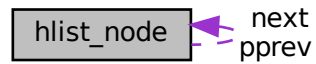
- struct `hlist_node` * **first**

The documentation for this struct was generated from the following file:

- `/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/list.h`

5.2 hlist_node Struct Reference

Collaboration diagram for hlist_node:



Data Fields

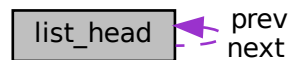
- struct [hlist_node](#) * **next**
- struct [hlist_node](#) ** **pprev**

The documentation for this struct was generated from the following file:

- [/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/list.h](#)

5.3 list_head Struct Reference

Collaboration diagram for list_head:



Data Fields

- struct [list_head](#) * **next**
- struct [list_head](#) * **prev**

The documentation for this struct was generated from the following file:

- [/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/list.h](#)

5.4 procfs_proc_umsdata Struct Reference

Structure defining the data needed by a process to manage its procfs instance.

```
#include <shared.h>
```

Data Fields

- struct proc_dir_entry * [proc_dir](#)
Process directory (/proc/ums/<pid>/).
- struct proc_dir_entry * [scheds_dir](#)
Schedulers directory (/proc/ums/<pid>/schedulers/).

5.4.1 Detailed Description

Structure defining the data needed by a process to manage its procfs instance.

The documentation for this struct was generated from the following file:

- /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/[shared.h](#)

5.5 procfs_sched_umsdata Struct Reference

Structure defining the procfs data of a scheduler.

```
#include <shared.h>
```

Data Fields

- struct proc_dir_entry * [sched_dir](#)
Scheduler's directory (/proc/ums/<pid>/schedulers/<id>/).
- struct proc_dir_entry * [workers_dir](#)
Workers directory (/proc/ums/<pid>/schedulers/<id>/workers/).
- struct proc_dir_entry * [info_file](#)
Scheduler's informations and statistics.
- unsigned long [num_switches](#)
The total number of context switches performed.
- ktime_t [last_switch](#)
The time required to perform the last context switch.

5.5.1 Detailed Description

Structure defining the procfs data of a scheduler.

The documentation for this struct was generated from the following file:

- /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/[shared.h](#)

5.6 procfs_worker_umsdata Struct Reference

Structure defining the procfs data of a worker.

```
#include <shared.h>
```

Data Fields

- ktime_t [tot_running](#)
The total running time of the worker.
- ktime_t [start_time](#)
The time which the worker started running.
- unsigned long [num_switches](#)
The total number of context switches performed.

5.6.1 Detailed Description

Structure defining the procfs data of a worker.

The documentation for this struct was generated from the following file:

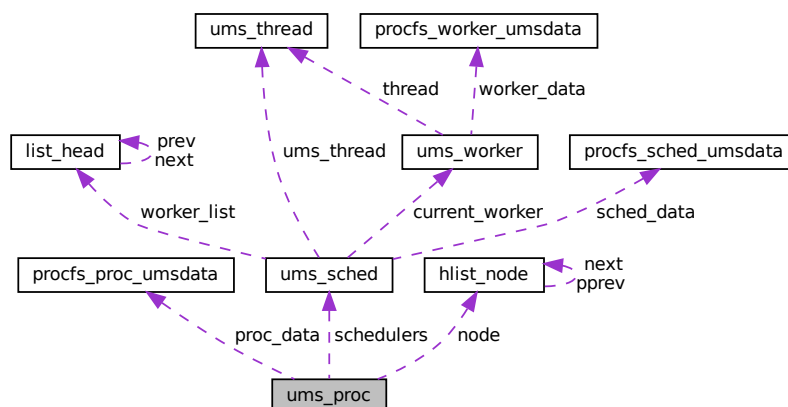
- </home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/shared.h>

5.7 ums_proc Struct Reference

Structure defining the components that identify a process that is using the UMS driver.

```
#include <shared.h>
```

Collaboration diagram for ums_proc:



Public Member Functions

- [DECLARE_HASHTABLE](#) (workers, HBITS)
Hashtable of all workers initilized by a process.

Data Fields

- struct [ums_sched](#) ** [schedulers](#)
An array of pointers to all the active scheduler threads.
- struct [hlist_node](#) [node](#)
Node element for placing the process in the global hashtable.
- [pid_t](#) [pid](#)
The process PID.
- [rwlock_t](#) [hash_lock](#)
Lock used to access the workers hashtable.
- [wait_queue_head_t](#) [wq](#)
A wait queue used to implement the blocking call `DequeueUmsCompletionListItems`.
- struct [procfs_proc_umsdata](#) * [proc_data](#)
The procfs data relative to the process.

5.7.1 Detailed Description

Structure defining the components that identify a process that is using the UMS driver.

The documentation for this struct was generated from the following file:

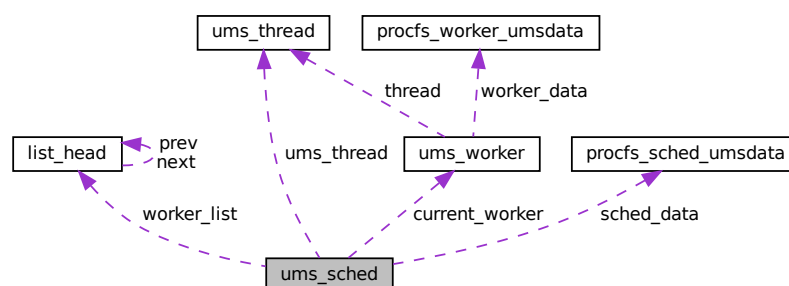
- `/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/shared.h`

5.8 ums_sched Struct Reference

Structure defining a scheduler thread in UMS mode.

```
#include <shared.h>
```

Collaboration diagram for ums_sched:



Public Member Functions

- [DECLARE_HASHTABLE](#) ([worker_list](#), HBITS)
The completion list (implemented as an hashtable).

Data Fields

- struct task_struct * [sched_task](#)
The task_struct of the scheduler thread.
- struct ums_worker * [current_worker](#)
The worker currently running on the scheduler context, NULL if none.
- struct pt_regs [sched_regs](#)
Backup of the scheduler's state, used to implement the context switch.
- rwlock_t [lock](#)
Lock used to access the hashtable.
- struct procs_sched_umsdata * [sched_data](#)
The procs data related to the scheduler.
- struct ums_thread * [ums_thread](#)
The UMS thread related to the scheduler.
- unsigned int [nworkers](#)
The total number of workers in the worker_list.
- struct list_head * [worker_list](#)
The completion list assigned to the scheduler.

5.8.1 Detailed Description

Structure defining a scheduler thread in UMS mode.

Structure defining a scheduler thread and its context.

5.8.2 Field Documentation

5.8.2.1 current_worker

```
struct ums_worker * ums_sched::current_worker
```

The worker currently running on the scheduler context, NULL if none.

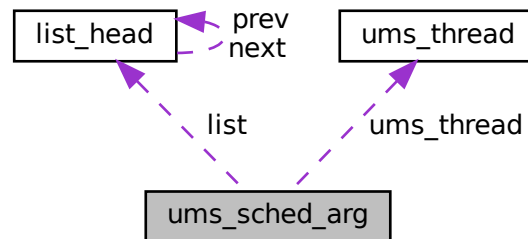
The worker currently executing in the scheduler's context.

The documentation for this struct was generated from the following file:

- /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/[shared.h](#)

5.9 ums_sched_arg Struct Reference

Collaboration diagram for ums_sched_arg:



Data Fields

- struct `ums_thread` * `ums_thread`
The UMS thread corresponding to the scheduler thread.
- void(* `sched_routine`)(void)
The scheduler function.
- struct `list_head` * `list`
- unsigned int `cpuid`
The head of the completion list.
- sem_t * `sem`
Semaphore used to coordinate the main thread with the scheduler thread.

5.9.1 Field Documentation

5.9.1.1 cpuid

```
unsigned int ums_sched_arg::cpuid
```

The head of the completion list.

The CPU to which the thread will be bound.

The documentation for this struct was generated from the following file:

- `/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/shared.h`

5.10 ums_thread Struct Reference

Structure defining a UMS specific thread.

```
#include <shared.h>
```

Data Fields

- pthread_t [pthread](#)
The related pthread.
- pid_t [tid](#)
The TID of the created thread.

5.10.1 Detailed Description

Structure defining a UMS specific thread.

The documentation for this struct was generated from the following file:

- /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/[shared.h](#)

5.11 ums_usr_worker Struct Reference

Structure defining a tuple used to hold registration information about a worker.

```
#include <shared.h>
```

Data Fields

- unsigned int [cpuid](#)
The cpuid related to the scheduler on which we are registering the worker.
- pid_t [tid](#)
The PID of the worker thread.

5.11.1 Detailed Description

Structure defining a tuple used to hold registration information about a worker.

Structure defining a tuple used to send worker thread registration informations to the UMS driver.

5.11.2 Field Documentation

5.11.2.1 cpuid

```
unsigned int ums_usr_worker::cpuid
```

The cpuid related to the scheduler on which we are registering the worker.

The cpuid of the scheduler on which we are registering the worker.

5.11.2.2 tid

```
pid_t ums_usr_worker::tid
```

The PID of the worker thread.

The TID of the target worker.

The documentation for this struct was generated from the following file:

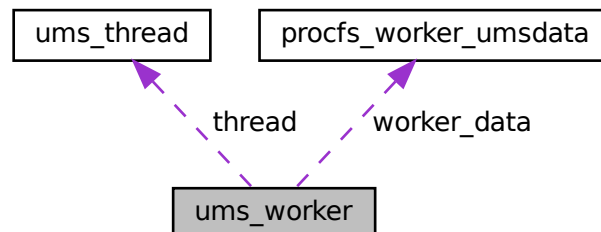
- /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/[shared.h](#)

5.12 ums_worker Struct Reference

Structure defining a worker thread.

```
#include <shared.h>
```

Collaboration diagram for ums_worker:



Data Fields

- struct task_struct * [task](#)
The task_struct of the worker thread.
- enum [state](#) [state](#)
The current state of the worker. (IDLE, TERMINATED, SCHEDULED, RUNNING)
- struct kref [refcnt](#)
Reference counter for the worker.
- rwlock_t [rwlock](#)
Lock used to keep coherent the state of a worker.
- struct procfs_worker_umdata * [worker_data](#)
The procfs data of the worker.
- struct ums_thread [thread](#)
The corresponding UMS thread.
- unsigned long [refcnt](#)
Reference counter, used to keep track of how many lists contain this worker.
- pthread_mutex_t [mutex](#)
Mutex to keep the worker's state coherent as well as the refcnt.

5.12.1 Detailed Description

Structure defining a worker thread.

Structure defining a worker.

5.12.2 Field Documentation

5.12.2.1 state

```
enum state ums_worker::state
```

The current state of the worker. (IDLE, TERMINATED, SCHEDULED, RUNNING)

The state of the worker.

The documentation for this struct was generated from the following file:

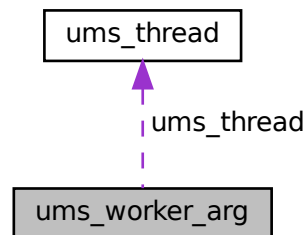
- /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/[shared.h](#)

5.13 ums_worker_arg Struct Reference

Structure used to wrap the arguments of pthread_create within ums_worker_create.

```
#include <shared.h>
```

Collaboration diagram for ums_worker_arg:



Data Fields

- struct `ums_thread` * `ums_thread`
The corresponding UMS specific thread.
- void(* `ums_routine`)(void *)
The routine passed to `ums_worker_create`.
- void * `arg`
The argument passed to `ums_worker_create`.
- sem_t * `tid_sem`
Semaphore used to coordinate the assignment of the TID in the struct `ums_thread`.

5.13.1 Detailed Description

Structure used to wrap the arguments of pthread_create within ums_worker_create.

The documentation for this struct was generated from the following file:

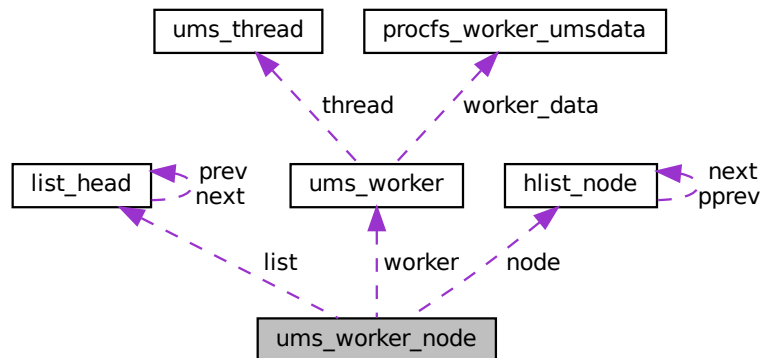
- `/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/shared.h`

5.14 ums_worker_node Struct Reference

Structure used to hold a shared worker in a hashtable.

```
#include <shared.h>
```

Collaboration diagram for ums_worker_node:



Data Fields

- struct `ums_worker` * `worker`
Worker assigned to the node.
- pid_t `tid`
The PID of the worker thread. It is used as a key for the hashtable.
- struct `hlist_node` `node`
Node element for placing the worker in a hashtable.
- struct `proc_dir_entry` * `info_file`
The directory entry that represents the node in ProcFS.
- struct `list_head` `list`
The member used to insert the node in a list.

5.14.1 Detailed Description

Structure used to hold a shared worker in a hashtable.

Structure used as a wrapper to have multiple nodes sharing the same worker.

The documentation for this struct was generated from the following file:

- `/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/shared.h`

Chapter 6

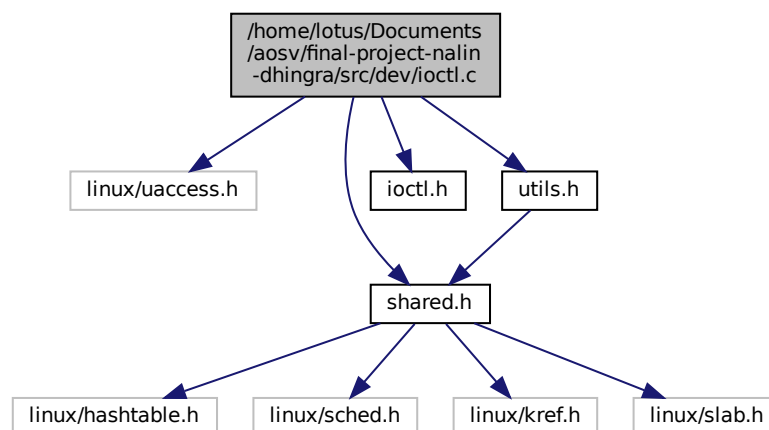
File Documentation

6.1 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ioctl.c File Reference

IOCTL API to interact with the UMS device.

```
#include <linux/uaccess.h>
#include "shared.h"
#include "ioctl.h"
#include "utils.h"
```

Include dependency graph for ioctl.c:



Functions

- long [ums_ioctl](#) (struct file *file, unsigned int cmd, unsigned long arg)

6.1.1 Detailed Description

IOCTL API to interact with the UMS device.

This file exposes the IOCTL API to make use of the functionalities of UMS in userspace.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.1.2 Function Documentation

6.1.2.1 ums_ioctl()

```
long ums_ioctl (
    struct file * file,
    unsigned int cmd,
    unsigned long arg )
```

IOCTL implementation to handle interaction with the LKM.

Parameters

<i>file</i>	The file associated with the device fd.
<i>cmd</i>	The command to be executed.
<i>arg</i>	A potential argument needed to execute the wanted command.

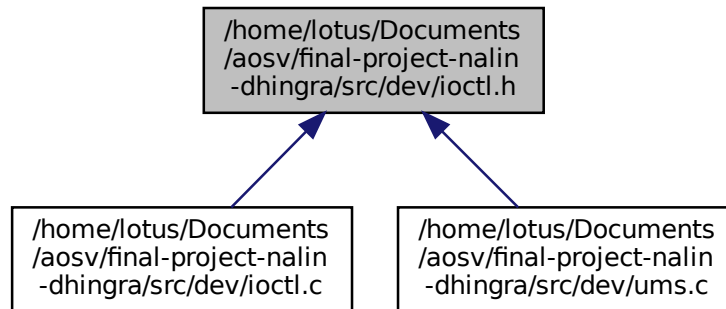
Returns

SUCCESS or an error (< 0).

6.2 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ioctl.h File Reference

Header file for the IOCTL API.

This graph shows which files directly or indirectly include this file:



Macros

- `#define INIT_WORKER _IO(0x1337, 'a')`
Command to initialize a worker within the module.
- `#define REGISTER_PROC _IO(0x1337, 'b')`
Command to register a process to be in UMS mode.
- `#define TERMINATE_PROC _IO(0x1337, 'c')`
Command to unregister a process that is in UMS mode.
- `#define REGISTER_SCHED _IOW(0x1337, 'd', unsigned int)`
Command to register a scheduler thread.
- `#define REGISTER_WORKER _IOW(0x1337, 'e', struct ums_usr_worker)`
Command to register a worker to a precise scheduler.
- `#define EXECUTE_THREAD _IOW(0x1337, 'f', pid_t)`
Command to execute a ums worker.
- `#define THREAD_YIELD _IO(0x1337, 'g')`
Command to yield the calling thread.
- `#define DEQUEUE_LIST _IOWR(0x1337, 'h', int)`
Command to get the list of available workers.
- `#define TERMINATE_WORKER _IO(0x1337, 'i')`
Command to terminate a running worker and restore the scheduler hosting it.

Functions

- long `ums_ioctl` (struct file *file, unsigned int cmd, unsigned long arg)

6.2.1 Detailed Description

Header file for the IOCTL API.

This header file contains the definitions of the command exposed in the IOCTL API for the UMS module.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.2.2 Macro Definition Documentation

6.2.2.1 DEQUEUE_LIST

```
#define DEQUEUE_LIST _IOWR(0x1337, 'h', int)
```

Command to get the list of available workers.

Parameters

<i>int</i>	This parameter is the pointer to an array, which holds as the first value the number of workers to be dequeued.
------------	---

6.2.2.2 EXECUTE_THREAD

```
#define EXECUTE_THREAD _IOW(0x1337, 'f', pid_t)
```

Command to execute a ums worker.

Parameters

<i>pid_t</i>	The PID of the worker thread to be executed.
--------------	--

6.2.2.3 INIT_WORKER

```
#define INIT_WORKER _IO(0x1337, 'a')
```

Command to initialize a worker within the module.

This command registers the worker thread to the list of the process in UMS mode and modifies the state of the thread to TASK_IDLE to avoid the kernel scheduler routine to execute the thread.

6.2.2.4 REGISTER_SCHED

```
#define REGISTER_SCHED _IOW(0x1337, 'd', unsigned int)
```

Command to register a scheduler thread.

Parameters

<code>unsigned_int</code>	The ID of the CPU on top of which the thread will run.
---------------------------	--

6.2.2.5 REGISTER_WORKER

```
#define REGISTER_WORKER _IOW(0x1337, 'e', struct ums_usr_worker)
```

Command to register a worker to a precise scheduler.

Parameters

<code>ums_usr_worker</code>	A tuple formed by the TID of the worker and the CPUID of the scheduler.
-----------------------------	---

6.2.3 Function Documentation**6.2.3.1 ums_ioctl()**

```
long ums_ioctl (
    struct file * file,
    unsigned int cmd,
    unsigned long arg )
```

IOCTL implementation to handle interaction with the LKM.

Parameters

<code>file</code>	The file associated with the device fd.
<code>cmd</code>	The command to be executed.
<code>arg</code>	A potential argument needed to execute the wanted command.

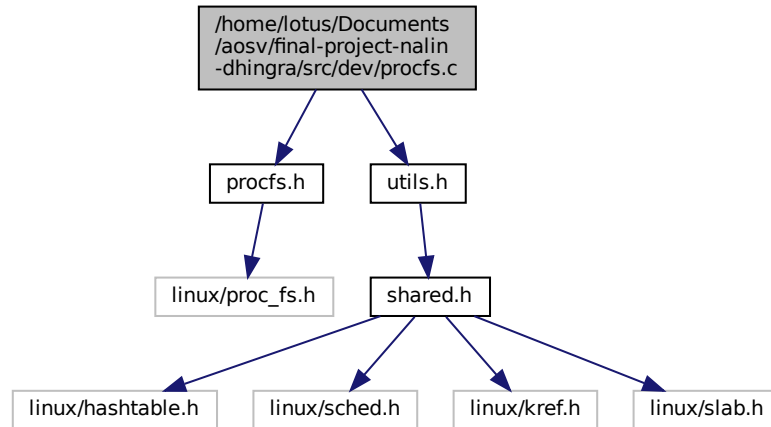
Returns

SUCCESS or an error (< 0).

6.3 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/procfs.c File Reference

Implementation of the ProcFS interface related to the UMS driver.

```
#include "procfs.h"
#include "utils.h"
Include dependency graph for procfs.c:
```



Functions

- `struct proc_dir_entry * init\_procfs` (void)
Initializes the procfs data for UMS LKM.
- `ssize_t sched\_proc\_read` (struct file *filp, char __user *buffer, size_t length, loff_t *offset)
Prints the data corresponding to the target scheduler.
- `ssize_t worker\_proc\_read` (struct file *filp, char __user *buffer, size_t length, loff_t *offset)
Prints the data corresponding to the target worker.

Variables

- `struct proc_dir_entry * procfs\_base\_dir`
The proc_dir_entry corresponding to the UMS dir entry, which is: `/proc/ums/`.

6.3.1 Detailed Description

Implementation of the ProcFS interface related to the UMS driver.

This file contains the functions and implementations to manage the Proc filesystem and expose the statistical data about the UMS driver.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.3.2 Function Documentation

6.3.2.1 sched_proc_read()

```
ssize_t sched_proc_read (
    struct file * filp,
    char __user * buffer,
    size_t length,
    loff_t * offset )
```

Prints the data corresponding to the target scheduler.

This functions retrieves the data of the scheduler binded to the file of the type: "/proc/ums/<PID>/schedulers/<ID>/info" and copies it to userspace.

6.3.2.2 worker_proc_read()

```
ssize_t worker_proc_read (
    struct file * filp,
    char __user * buffer,
    size_t length,
    loff_t * offset )
```

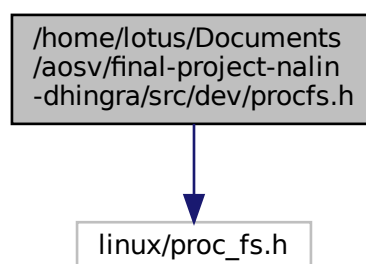
Prints the data corresponding to the target worker.

This functions retrieves the data of the worker binded to the file of the type: "/proc/ums/<PID>/schedulers/<ID>/workers/<TID>" and copies it to userspace.

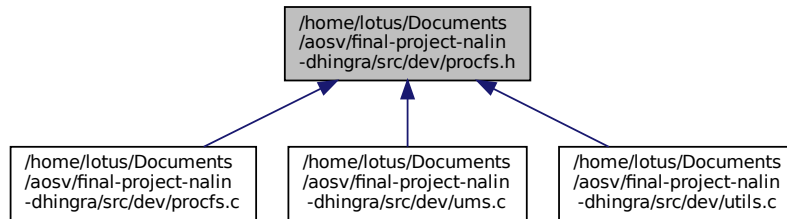
6.4 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/procfs.h File Reference

Header file for the ProcFS interface of the UMS driver.

```
#include <linux/proc_fs.h>
Include dependency graph for procfs.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define LINUX_KERNEL_VERSION 510`
Defines the version of the kernel in use (5.10.122).

Functions

- `struct proc_dir_entry * init_procfs (void)`
Initializes the procfs data for UMS LKM.
- `ssize_t sched_proc_read (struct file *filp, char __user *buffer, size_t length, loff_t *offset)`
Prints the data corresponding to the target scheduler.
- `ssize_t worker_proc_read (struct file *filp, char __user *buffer, size_t length, loff_t *offset)`
Prints the data corresponding to the target worker.

Variables

- `struct proc_dir_entry * procfs_base_dir`
The proc_dir_entry corresponding to the UMS dir entry, which is: "/proc/ums/".

6.4.1 Detailed Description

Header file for the ProcFS interface of the UMS driver.

This file contains the prototypes of the functions needed to interact with the ProcFS.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.4.2 Function Documentation

6.4.2.1 sched_proc_read()

```
ssize_t sched_proc_read (
    struct file * filp,
    char __user * buffer,
    size_t length,
    loff_t * offset )
```

Prints the data corresponding to the target scheduler.

This functions retrieves the data of the scheduler binded to the file of the type: "/proc/ums/<PID>/schedulers/<ID>/info" and copies it to userspace.

6.4.2.2 worker_proc_read()

```
ssize_t worker_proc_read (
    struct file * filp,
    char __user * buffer,
    size_t length,
    loff_t * offset )
```

Prints the data corresponding to the target worker.

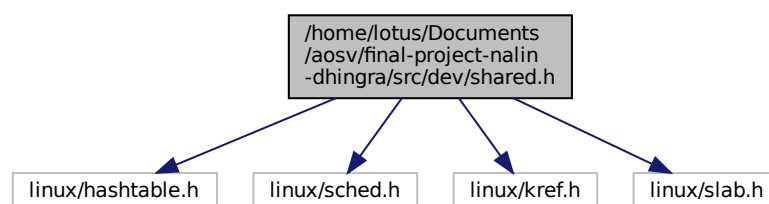
This functions retrieves the data of the worker binded to the file of the type: "/proc/ums/<PID>/schedulers/<ID>/workers/<TID>" and copies it to userspace.

6.5 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/shared.h File Reference

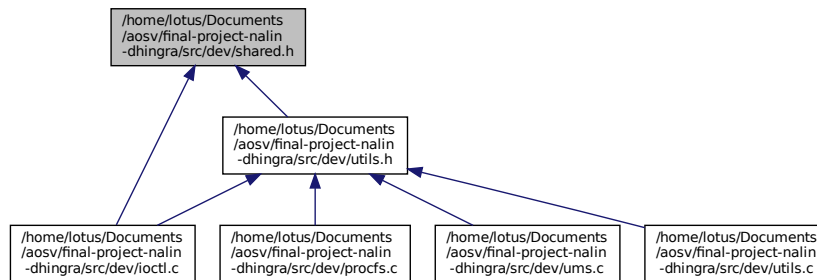
File used to define global variables declared in shared.h.

```
#include <linux/hashtable.h>
#include <linux/sched.h>
#include <linux/kref.h>
#include <linux/slab.h>
```

Include dependency graph for shared.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [ums_proc](#)
Structure defining the components that identify a process that is using the UMS driver.
- struct [ums_sched](#)
Structure defining a scheduler thread in UMS mode.
- struct [ums_worker](#)
Structure defining a worker thread.
- struct [ums_worker_node](#)
Structure used to hold a shared worker in a hashtable.
- struct [ums_usr_worker](#)
Structure defining a tuple used to hold registration information about a worker.
- struct [procfs_proc_umsdata](#)
Structure defining the data needed by a process to manage its procfs instance.
- struct [procfs_sched_umsdata](#)
Structure defining the procfs data of a scheduler.
- struct [procfs_worker_umsdata](#)
Structure defining the procfs data of a worker.

Macros

- #define [DEVICE_NAME](#) "umsdev"
The device name in "dev".
- #define [LOG_MSG](#) "umsdev: "
Log message to identify driver's messages in dmesg.
- #define **SUCCESS** 0
- #define **FAILURE** -1
- #define [HBITS](#) 8
"bits" used by a general hashtable (8 = 256 entries).
- #define **PRINTDBG**(fmt, args...) pr_debug([LOG_MSG](#) "CPU[%d]" fmt "\n", current->cpu, ##args)

Typedefs

- typedef struct [ums_worker_node](#) [ums_worker_node_t](#)
Structure used to hold a shared worker in a hashtable.

Enumerations

- enum [state](#) {
 WORKER_RUNNING, **WORKER_IDLE**, **WORKER_TERMINATED**, **WORKER_SCHEDULED**,
 WORKER_RUNNING, **WORKER_IDLE**, **WORKER_TERMINATED** }

Representation of the possible states a UMS worker can be in.

Functions

- [DECLARE_HASHTABLE](#) (ums_procs, [HBITS](#))

Hashtable to keep all the processes that are registered in UMS mode.

Variables

- int [ncpus](#)

The number of online CPUs in the system.

6.5.1 Detailed Description

File used to define global variables declared in shared.h.

Header file containing shared data between most LKM files.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

This file contains the informations and data needed by all the files of the LKM.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.5.2 Variable Documentation

6.5.2.1 ncpus

```
int ncpus
```

The number of online CPUs in the system.

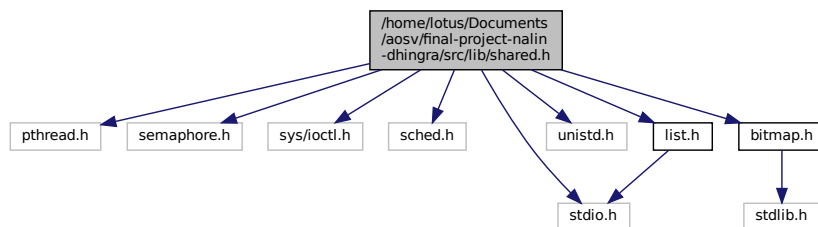
The number of online CPUs in the system.

6.6 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/shared.h File Reference

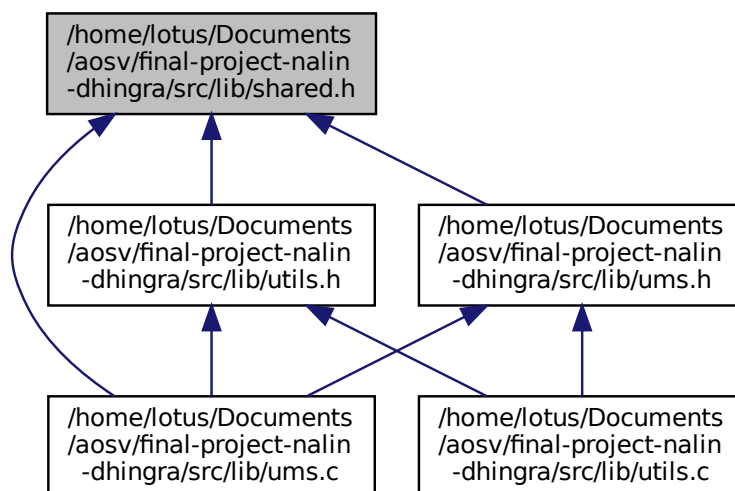
Contains data shared among most of the files of the UMS library.

```
#include <pthread.h>
#include <semaphore.h>
#include <sys/ioctl.h>
#include <sched.h>
#include <stdio.h>
#include <unistd.h>
#include "bitmap.h"
#include "list.h"
```

Include dependency graph for shared.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [ums_thread](#)
Structure defining a UMS specific thread.
- struct [ums_worker_arg](#)
Structure used to wrap the arguments of pthread_create within ums_worker_create.
- struct [ums_sched_arg](#)
- struct [ums_worker](#)
Structure defining a worker thread.
- struct [ums_worker_node](#)
Structure used to hold a shared worker in a hashtable.
- struct [ums_sched](#)
Structure defining a scheduler thread in UMS mode.
- struct [ums_usr_worker](#)
Structure defining a tuple used to hold registration information about a worker.

Macros

- #define **SUCCESS** 0
- #define **FAILURE** -1
- #define [DRIVER_PATH](#) "/dev/umsdev"
File path of the IOCTL driver.
- #define **PRINTDBG**(...) do {} while(0)
- #define [INIT_WORKER](#) _IO(0x1337, 'a')
Command to initialize the thread of the calling worker in UMS mode.
- #define [REGISTER_PROC](#) _IO(0x1337, 'b')
Command to register a process to be in UMS mode.
- #define [TERMINATE_PROC](#) _IO(0x1337, 'c')
Command to unregister a process that is in UMS mode.
- #define [REGISTER_SCHED](#) _IOW(0x1337, 'd', unsigned int)
Command to register a scheduler thread.
- #define [REGISTER_WORKER](#) _IOW(0x1337, 'e', struct [ums_usr_worker](#))
Command to register a worker to a precise scheduler.
- #define [EXECUTE_THREAD](#) _IOW(0x1337, 'f', pid_t)
Command to execute a ums worker.
- #define [THREAD_YIELD](#) _IO(0x1337, 'g')
Command to yield the calling worker thread.
- #define [DEQUEUE_LIST](#) _IOWR(0x1337, 'h', int)
Command to get the list of available workers.
- #define [TERMINATE_WORKER](#) _IO(0x1337, 'i')
Command to terminate a running worker and restore the scheduler hosting it.

Typedefs

- typedef struct [ums_worker_node](#) [ums_worker_node_t](#)
Structure used as a wrapper to have multiple nodes sharing the same worker.

Enumerations

- enum **state** {
WORKER_RUNNING, **WORKER_IDLE**, **WORKER_TERMINATED**, **WORKER_SCHEDULED**,
WORKER_RUNNING, **WORKER_IDLE**, **WORKER_TERMINATED** }

Variables

- int **dev_fd**
The file descriptor of "/dev/umsdev".
- size_t **ncpus**
The number of available CPUs in the system.
- bitmap_t **cpus_map**
A bitmap representing the state of the CPUs in the UMS context of this process.
- struct **ums_sched** ** **ums_schedulers**
An array of pointers to the schedulers in use.
- struct **list_head** **global_list**
A list containing all the created UMS workers in the process context.

6.6.1 Detailed Description

Contains data shared among most of the files of the UMS library.

This file contains the headers, macros, data structures and global variables that need to be shared among all files.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.6.2 Variable Documentation

6.6.2.1 ncpus

```
size_t ncpus
```

The number of available CPUs in the system.

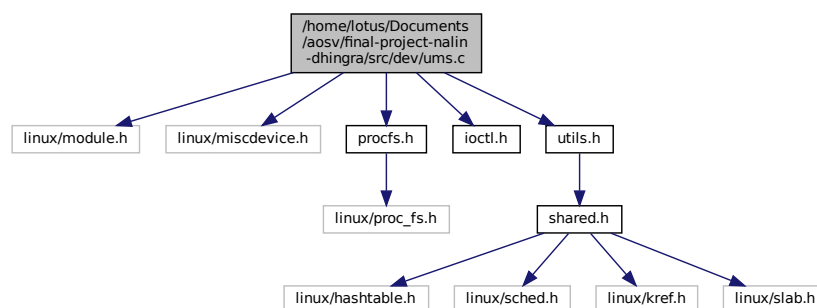
The number of online CPUs in the system.

6.7 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/ums.c File Reference

Initialization of LKM.

```
#include <linux/module.h>
#include <linux/miscdevice.h>
#include "procfs.h"
#include "ioctl.h"
#include "utils.h"
```

Include dependency graph for ums.c:



6.7.1 Detailed Description

Initialization of LKM.

This file Manages the initialization of the Linux Kernel module for the UMS driver.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

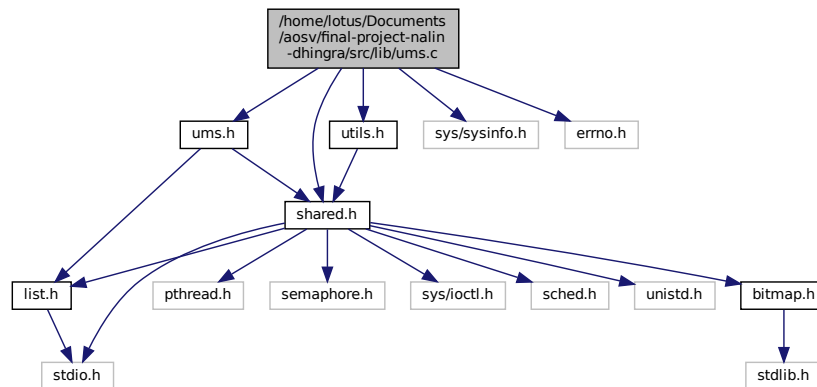
6.8 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.c File Reference

Implements the main user API to use the funtionalities of the UMS driver.

```
#include "ums.h"
#include "shared.h"
#include "utils.h"
#include <sys/sysinfo.h>
```

```
#include <errno.h>
```

Include dependency graph for ums.c:



Functions

- `int ums_init ()`
Initializes the UMS driver for the current process.
- `void ums_destroy ()`
Cleans up all UMS data created by a process.
- `int ums_worker_create (struct ums_worker *worker, void(*start_routine)(void *), void *arg)`
Creates a UMS worker.
- `int ums_worker_list_insert (struct list_head *head, struct ums_worker *worker)`
Inserts a UMS worker into a completion list.
- `int EnterUmsSchedulingMode (void(*scheduler_routine)(), struct list_head *worker_list)`
Initializes the scheduler thread with its relative worker list.
- `int ExecuteUmsThread (struct ums_worker *worker)`
Executes the given worker in the scheduler's context.
- `int UmsThreadYield (void)`
Yields the current worker and restores the scheduler's context.
- `struct list_head * DequeueUmsCompletionListItems (size_t nworkers)`
Returns a list of valid workers that are available to be executed.

6.8.1 Detailed Description

Implements the main user API to use the functionalities of the UMS driver.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.8.2 Function Documentation

6.8.2.1 DequeueUmsCompletionListItems()

```
struct list_head* DequeueUmsCompletionListItems (
    size_t nworkers )
```

Returns a list of valid workers that are available to be executed.

This function is a blocking function and will return a sublist of workers from the scheduler's completion list, that are available to be executed on this scheduler.

Parameters

<i>nworkers</i>	The number of workers wanted in the queue. If the value is either equal to 0 or greater then the number available workers, the latter value will be used instead.
-----------------	---

Returns

struct [list_head](#) *: A pointer to a list head that will contain valid workers. The list has to be freed by the user.

A pointer to a struct [list_head](#) representing the dequeued list or NULL if there are no more workers to be executed.

6.8.2.2 EnterUmsSchedulingMode()

```
int EnterUmsSchedulingMode (
    void(*)() scheduler_routine,
    struct list_head * worker_list )
```

Initializes the scheduler thread with its relative worker list.

This function creates a scheduler thread and assigns it to the first non used CPU for this process's UMS. If there are no more CPUs available to the process it will return an error. It also assigns the given worker_list to that scheduler, performing additional operations to register both components to the UMS driver. This function must only be called by the main thread, any other usage, may cause unwanted behaviour.

Parameters

<i>scheduler_routine</i>	The scheduling function that will be executed by the scheduler thread.
<i>worker_list</i>	The worker list to be assigned to the scheduler.

Returns

0 if successful <0 if failed.

6.8.2.3 ExecuteUmsThread()

```
int ExecuteUmsThread (
    struct ums_worker * worker )
```

Executes the given worker in the scheduler's context.

Parameters

<i>worker</i>	The worker to be executed.
---------------	----------------------------

Returns

0 if successful <0 if failed.

6.8.2.4 ums_destroy()

```
void ums_destroy (
    void )
```

Cleans up all UMS data created by a process.

This function is to be called when a process is terminating its UMS session. Calling this function when a thread is still running may result in unexpected behaviour.

6.8.2.5 ums_init()

```
int ums_init (
    void )
```

Initializes the UMS driver for the current process.

This function is to be called by the main thread before starting a UMS session.

Returns

0 if successful <0 if failed.

6.8.2.6 ums_worker_create()

```
int ums_worker_create (
    struct ums_worker * worker,
    void(*) (void *) start_routine,
    void * arg )
```

Creates a UMS worker.

This function is used as a wrapper for `pthread_create`, to instantiate a UMS thread.

Parameters

<i>worker</i>	A pointer to a struct <code>ums_worker</code> that will be initialized.
<i>start_routine</i>	The routine that will be executed by the UMS thread.
<i>arg</i>	The argument needed by the UMS thread routine.

Returns

0 if successful <0 if failed.

6.8.2.7 `ums_worker_list_insert()`

```
int ums_worker_list_insert (
    struct list_head * head,
    struct ums_worker * worker )
```

Inserts a UMS worker into a completion list.

Parameters

<i>head</i>	The head of the list.
<i>worker</i>	The worker to be inserted as a node.

Returns

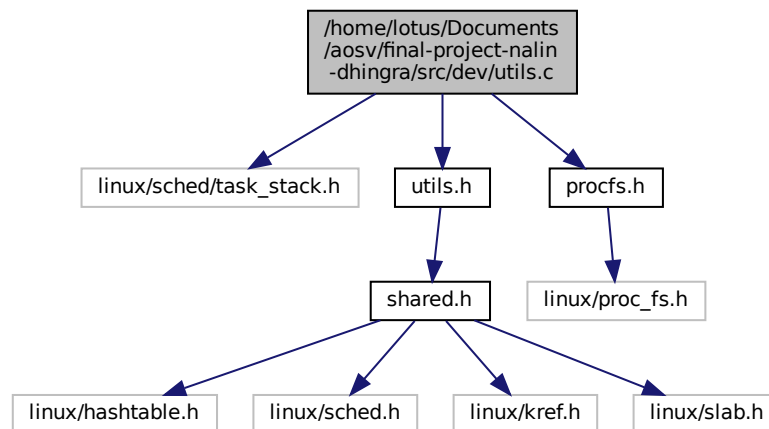
0 if successful <0 if failed.

6.9 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/utils.c File Reference

Helper functions and data structures used by LKM.

```
#include <linux/sched/task_stack.h>
#include "utils.h"
#include "procfs.h"
```

Include dependency graph for `utils.c`:



Functions

- `struct ums_proc * find_ums_proc (pid_t pid)`
Finds a UMS process given its PID.
- `long register_ums_process (pid_t pid)`
Registers the given process in UMS mode.
- `long terminate_ums_process (pid_t pid)`
Unregisters the given process in UMS mode.
- `long cleanup_process (struct ums_proc *process)`
Cleanup the data structures used by a process.
- `void worker_release (struct kref *refcnt)`
Routine used to free worker when releasing the last kref.
- `long init_worker_node (void)`
Initializes a worker node and saves it in the given process hashtable.
- `long register_ums_scheduler (unsigned int cpuid)`
Registers a new scheduler for the target process to be run on a specific cpu.
- `long register_usr_worker (struct ums_usr_worker *usr_worker)`
Registers a worker thread on a specified scheduler's worker list.
- `long execute_thread (pid_t tid)`
Executes the given worker on the current cpu.
- `void thread_yield ()`
Yields the current running worker and restores the scheduler's context.
- `void dequeue_list (size_t size, unsigned int *tid_list)`
Populates the array with the available worker's using their TIDs.
- `void terminate_worker (void)`
Terminates the current worker and restores the scheduler hosting it.

6.9.1 Detailed Description

Helper functions and data structures used by LKM.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.9.2 Function Documentation

6.9.2.1 cleanup_process()

```
long cleanup_process (
    struct ums_proc * process )
```

Cleanup the data structures used by a process.

Parameters

<i>process</i>	The struct <code>ums_proc</code> representing the process to cleanup.
----------------	---

Returns

SUCCESS or an error (<0).

6.9.2.2 dequeue_list()

```
void dequeue_list (
    size_t size,
    unsigned int * tid_list )
```

Populates the array with the available worker's using their TIDs.

Parameters

<i>size</i>	The size of the array.
<i>tid_list</i>	The array to be populated.

6.9.2.3 execute_thread()

```
long execute_thread (
    pid_t tid )
```

Executes the given worker on the current cpu.

Switches the execution from the scheduler thread to a worker thread, by swapping their pt_regs. This allows the scheduler's to effectively host the thread's execution.

Parameters

<i>tid</i>	The PID of the worker to be executed.
------------	---------------------------------------

Returns

SUCCESS or an error (<0).

6.9.2.4 find_ums_proc()

```
struct ums_proc* find_ums_proc (
    pid_t pid )
```

Finds a UMS process given its PID.

Parameters

<i>pid</i>	The pid of the process that holds resources (TGID).
------------	---

Returns

struct [ums_proc](#): The wanted [ums_proc](#) struct.

A pointer to a struct [ums_proc](#) or NULL if the process is not in the UMS processes hashtable.

6.9.2.5 init_worker_node()

```
long init_worker_node (
    void )
```

Initializes a worker node and saves it in the given process hashtable.

This function allocates the necessary data structures to represent a UMS worker and inserts it in the hashtable of the given process.

Returns

SUCCESS or an error (<0).

6.9.2.6 register_ums_process()

```
long register_ums_process (
    pid_t pid )
```

Registers the given process in UMS mode.

Parameters

<i>pid</i>	The pid of the process that holds resources (TGID).
------------	---

Returns

SUCCESS or an error (<0).

6.9.2.7 register_ums_scheduler()

```
long register_ums_scheduler (
    unsigned int cpuid )
```

Registers a new scheduler for the target process to be run on a specific cpu.

This function allocates the necessary memory to a struct [ums_sched](#), and initializes it according to the function's parameters.

Parameters

<i>cpuid</i>	The ID of the CPU on top of which the scheduler thread will run.
--------------	--

Returns

SUCCESS or an error (<0).

6.9.2.8 register_usr_worker()

```
long register_usr_worker (
    struct ums_usr_worker * usr_worker )
```

Registers a worker thread on a specified scheduler's worker list.

Parameters

<i>usr_worker</i>	The worker's informations.
-------------------	----------------------------

Returns

SUCCESS or an error (<0).

6.9.2.9 terminate_ums_process()

```
long terminate_ums_process (
    pid_t pid )
```

Unregisters the given process in UMS mode.

Parameters

<i>pid</i>	The pid of the process that holds resources (TGID).
------------	---

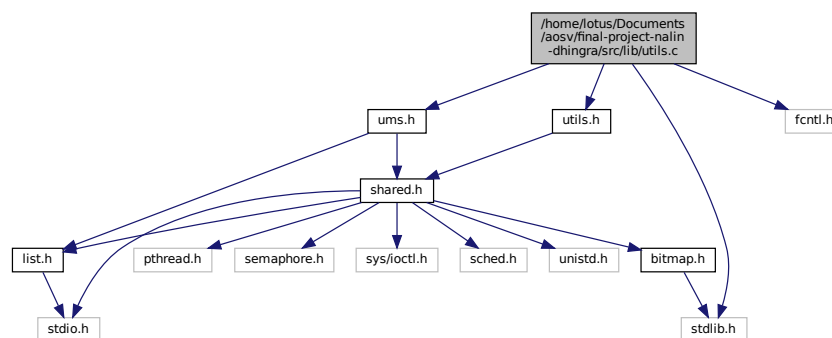
Returns

SUCCESS or an error (<0).

6.10 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/utils.c File Reference

Contains the helper functions for the UMS library.

```
#include "ums.h"
#include "utils.h"
#include <stdlib.h>
#include <fcntl.h>
Include dependency graph for utils.c:
```



Functions

- int `open_device` ()
Opens the driver's device.
- void * `worker_wrap_routine` (void *arg)
Wraps the original ums pthread routine for a worker thread.
- void * `sched_wrap_routine` (void *arg)
Wraps the original scheduler function.
- int `find_next_zero_bit` (unsigned long *map, size_t size)
Finds the first occurrence of a bit set to 0 in a bitmap.
- struct `ums_worker` * `find_worker_tid` (struct `list_head` *head, pid_t tid)
Finds the worker node corresponding to the given tid.
- struct `list_head` * `dup_worker_list` (struct `list_head` *head)
Duplicates the given completion list.

6.10.1 Detailed Description

Contains the helper functions for the UMS library.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.10.2 Function Documentation

6.10.2.1 `dup_worker_list()`

```
struct list_head* dup_worker_list (
    struct list_head * head )
```

Duplicates the given completion list.

Parameters

<i>head</i>	The head of the original list that needs to be duplicated.
-------------	--

Returns

A pointer to the new head of the duplicated completion list.

6.10.2.2 find_next_zero_bit()

```
int find_next_zero_bit (
    unsigned long * map,
    size_t size )
```

Finds the first occurrence of a bit set to 0 in a bitmap.

Parameters

<i>map</i>	The target bitmap.
<i>size</i>	The size of the bitmap.

Returns

The index (starting from 0) of the wanted bit or -1 if none were found.

6.10.2.3 find_worker_tid()

```
struct ums_worker* find_worker_tid (
    struct list_head * head,
    pid_t tid )
```

Finds the worker node corresponding to the given tid.

Parameters

<i>head</i>	The head of the list to search.
<i>tid</i>	The tid of the worker to be found.

Returns

A pointer to the wanted worker or NULL if there is no worker with such tid.

6.10.2.4 open_device()

```
int open_device (
    void )
```

Opens the driver's device.

Returns

int Returns a file descriptor to the open device or the relative error, if it could not open it.

6.10.2.5 sched_wrap_routine()

```
void* sched_wrap_routine (
    void * arg )
```

Wraps the original scheduler function.

Parameters

<i>arg</i>	Contains a struct ums_sched_arg , to perform scheduler initialization.
------------	--

6.10.2.6 worker_wrap_routine()

```
void* worker_wrap_routine (
    void * arg )
```

Wraps the original ums pthread routine for a worker thread.

This function is used to wrap the original ums_worker_create start_routine to correctly populate the needed data structures and to set the thread into an IDLE state, ready to be scheduled.

Parameters

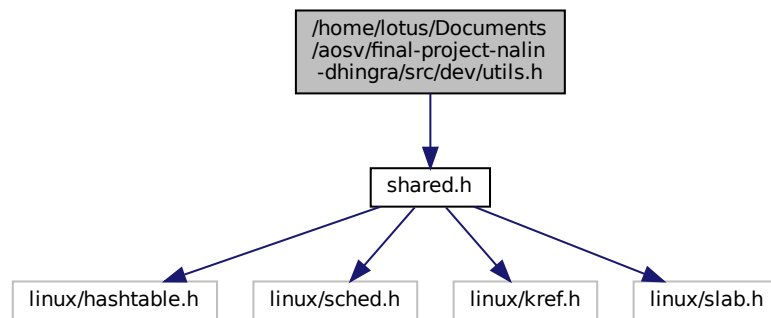
<i>arg</i>	A wrapper argument (of type struct ums_worker_arg) to the original *arg.
------------	---

6.11 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/utils.h File Reference

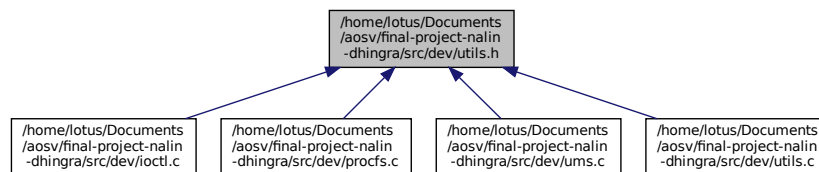
Headers for helper functions and data structures of LKM.

```
#include "shared.h"
```

Include dependency graph for utils.h:



This graph shows which files directly or indirectly include this file:



Functions

- struct `ums_proc` * `find_ums_proc` (pid_t pid)
Finds a UMS process given its PID.
- long `register_ums_process` (pid_t pid)
Registers the given process in UMS mode.
- long `terminate_ums_process` (pid_t pid)
Unregisters the given process in UMS mode.
- long `cleanup_process` (struct `ums_proc` *process)
Cleanup the data structures used by a process.
- long `init_worker_node` (void)
Initializes a worker node and saves it in the given process hashtable.
- long `register_ums_scheduler` (unsigned int cpuid)
Registers a new scheduler for the target process to be run on a specific cpu.
- long `register_usr_worker` (struct `ums_usr_worker` *usr_worker)
Registers a worker thread on a specified scheduler's worker list.
- long `execute_thread` (pid_t tid)
Executes the given worker on the current cpu.
- void `thread_yield` (void)
Yields the current running worker and restores the scheduler's context.
- void `dequeue_list` (size_t size, unsigned int *tid_list)

Populates the array with the available worker's using their TIDs.

- void [terminate_worker](#) (void)

Terminates the current worker and restores the scheduler hosting it.

- void [worker_release](#) (struct kref *refcnt)

Routine used to free worker when releasing the last kref.

6.11.1 Detailed Description

Headers for helper functions and data structures of LKM.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.11.2 Function Documentation

6.11.2.1 cleanup_process()

```
long cleanup_process (
    struct ums\_proc * process )
```

Cleanup the data structures used by a process.

Parameters

<i>process</i>	The struct ums_proc representing the process to cleanup.
----------------	--

Returns

SUCCESS or an error (<0).

6.11.2.2 dequeue_list()

```
void dequeue_list (
    size_t size,
    unsigned int * tid_list )
```

Populates the array with the available worker's using their TIDs.

Parameters

<i>size</i>	The size of the array.
<i>tid_list</i>	The array to be populated.

6.11.2.3 execute_thread()

```
long execute_thread (
    pid_t tid )
```

Executes the given worker on the current cpu.

Switches the execution from the scheduler thread to a worker thread, by swapping their pt_regs. This allows the scheduler's to effectively host the thread's execution.

Parameters

<i>tid</i>	The PID of the worker to be executed.
------------	---------------------------------------

Returns

SUCCESS or an error (<0).

6.11.2.4 find_ums_proc()

```
struct ums_proc* find_ums_proc (
    pid_t pid )
```

Finds a UMS process given its PID.

Parameters

<i>pid</i>	The pid of the process that holds resources (TGID).
------------	---

Returns

struct [ums_proc](#): The wanted [ums_proc](#) struct.

A pointer to a struct [ums_proc](#) or NULL if the process is not in the UMS processes hashtable.

6.11.2.5 init_worker_node()

```
long init_worker_node (
    void )
```

Initializes a worker node and saves it in the given process hashtable.

This function allocates the necessary data structures to represent a UMS worker and inserts it in the hashtable of the given process.

Returns

SUCCESS or an error (<0).

6.11.2.6 register_ums_process()

```
long register_ums_process (  
    pid_t pid )
```

Registers the given process in UMS mode.

Parameters

<i>pid</i>	The pid of the process that holds resources (TGID).
------------	---

Returns

SUCCESS or an error (<0).

6.11.2.7 register_ums_scheduler()

```
long register_ums_scheduler (  
    unsigned int cpuid )
```

Registers a new scheduler for the target process to be run on a specific cpu.

This function allocates the necessary memory to a struct [ums_sched](#), and initializes it according to the function's parameters.

Parameters

<i>cpuid</i>	The ID of the CPU on top of which the scheduler thread will run.
--------------	--

Returns

SUCCESS or an error (<0).

6.11.2.8 register_usr_worker()

```
long register_usr_worker (
    struct ums_usr_worker * usr_worker )
```

Registers a worker thread on a specified scheduler's worker list.

Parameters

<i>usr_worker</i>	The worker's informations.
-------------------	----------------------------

Returns

SUCCESS or an error (<0).

6.11.2.9 terminate_ums_process()

```
long terminate_ums_process (
    pid_t pid )
```

Unregisters the given process in UMS mode.

Parameters

<i>pid</i>	The pid of the process that holds resources (TGID).
------------	---

Returns

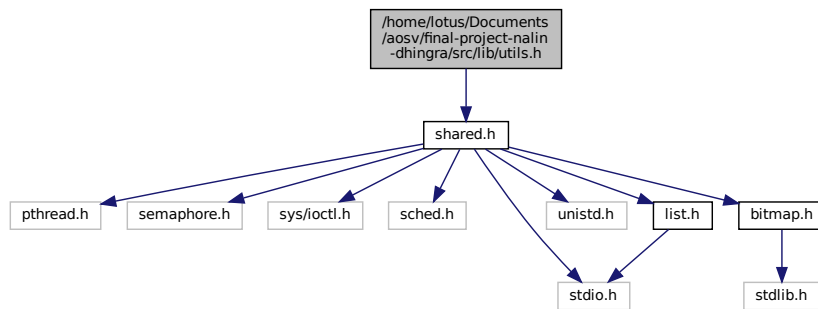
SUCCESS or an error (<0).

6.12 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/utils.h File Reference

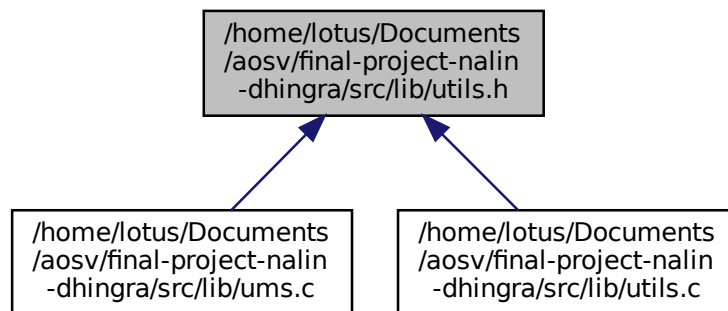
All helper functions for ums library.

```
#include "shared.h"
```

Include dependency graph for utils.h:



This graph shows which files directly or indirectly include this file:



Functions

- `int open_device (void)`
Opens the driver's device.
- `void * worker_wrap_routine (void *arg)`
Wraps the original ums pthread routine for a worker thread.
- `void * sched_wrap_routine (void *arg)`
Wraps the original scheduler function.
- `int find_next_zero_bit (unsigned long *map, size_t size)`
Finds the first occurrence of a bit set to 0 in a bitmap.
- `struct ums_worker * find_worker_tid (struct list_head *head, pid_t tid)`
Finds the worker node corresponding to the given tid.
- `struct list_head * dup_worker_list (struct list_head *head)`
Duplicates the given completion list.

6.12.1 Detailed Description

All helper functions for ums library.

This file contains the declarations of the functions and data structures needed by the main UMS library.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.12.2 Function Documentation

6.12.2.1 dup_worker_list()

```
struct list_head* dup_worker_list (  
    struct list_head * head )
```

Duplicates the given completion list.

Parameters

<i>head</i>	The head of the original list that needs to be duplicated.
-------------	--

Returns

A pointer to the new head of the duplicated completion list.

6.12.2.2 find_next_zero_bit()

```
int find_next_zero_bit (  
    unsigned long * map,  
    size_t size )
```

Finds the first occurrence of a bit set to 0 in a bitmap.

Parameters

<i>map</i>	The target bitmap.
<i>size</i>	The size of the bitmap.

Returns

The index (starting from 0) of the wanted bit or -1 if none were found.

6.12.2.3 find_worker_tid()

```
struct ums_worker* find_worker_tid (
    struct list_head * head,
    pid_t tid )
```

Finds the worker node corresponding to the given tid.

Parameters

<i>head</i>	The head of the list to search.
<i>tid</i>	The tid of the worker to be found.

Returns

A pointer to the wanted worker or NULL If there is no worker with such tid.

6.12.2.4 open_device()

```
int open_device (
    void )
```

Opens the driver's device.

Returns

int Returns a file descriptor to the open device or the relative error, if it could not open it.

6.12.2.5 sched_wrap_routine()

```
void* sched_wrap_routine (
    void * arg )
```

Wraps the original scheduler function.

Parameters

<i>arg</i>	Contains a struct <code>ums_sched_arg</code> , to perform scheduler initialization.
------------	---

6.12.2.6 worker_wrap_routine()

```
void* worker_wrap_routine (  
    void * arg )
```

Wraps the original ums pthread routine for a worker thread.

This function is used to wrap the original ums_worker_create start_routine to correctly populate the needed data structures and to set the thread into an IDLE state, ready to be scheduled.

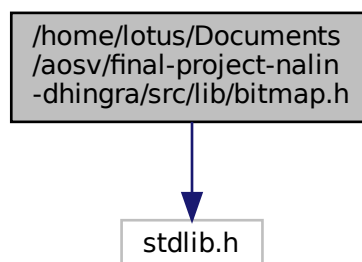
Parameters

<i>arg</i>	A wrapper argument (of type struct ums_worker_arg) to the original *arg.
------------	---

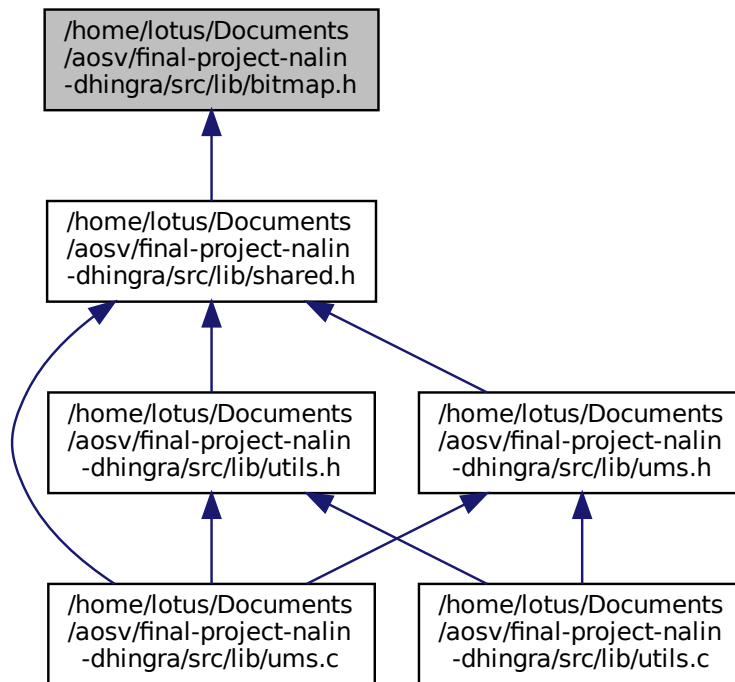
6.13 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/bitmap.h File Reference

Macros to implement bitmaps.

```
#include <stdlib.h>  
Include dependency graph for bitmap.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define BITS_TO_LONG(nr) ( (unsigned long)((nr + 7) / 8) )`
- `#define BITS_PER_LONG 64`
- `#define DECLARE_BITMAP(n) ( calloc(BITS_TO_LONG(n), sizeof(unsigned long)) )`
Creates a `bitmap_t` of n bits.
- `#define SET_BIT(b, i) ( b[i / BITS_PER_LONG] |= (unsigned long)(1 << (i & (BITS_PER_LONG-1))) )`
Sets the i^{th} bit of the given bitmap.
- `#define UNSET_BIT(b, i) ( b[i / BITS_PER_LONG] &= (unsigned long)~(1 << (i & (BITS_PER_LONG-1))) )`
Unsets the i^{th} bit of the given bitmap.
- `#define GET_BIT(b, i) ( b[i / BITS_PER_LONG] & (unsigned long)(1 << (i & (BITS_PER_LONG-1))) ? 1 : 0 )`
Gets the i^{th} bit of the given bitmap.

Typedefs

- `typedef unsigned long * bitmap_t`

6.13.1 Detailed Description

Macros to implement bitmaps.

This file contains the macros needed to create and manage bitmaps. This is inspired by the linux kernel implementation, but it is naive and doesn't enforce security.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.13.2 Macro Definition Documentation

6.13.2.1 DECLARE_BITMAP

```
#define DECLARE_BITMAP(  
    n ) ( calloc(BITS_TO_LONG(n), sizeof(unsigned long)) )
```

Creates a `bitmap_t` of `n` bits.

Returns a pointer to an array that needs to be freed, when not needed anymore.

Parameters

<i>n</i>	Number of bits needed in the bitmap.
----------	--------------------------------------

6.13.2.2 GET_BIT

```
#define GET_BIT(  
    b,  
    i ) ( b[i / BITS_PER_LONG] & (unsigned long)(1 << (i & (BITS_PER_LONG-1))) ? 1  
: 0 )
```

Gets the i^{th} bit of the given bitmap.

Parameters

<i>b</i>	The target bitmap.
<i>i</i>	The bit to be returned.

6.13.2.3 SET_BIT

```
#define SET_BIT(  
    b,  
    i ) ( b[i / BITS_PER_LONG] |= (unsigned long)(1 << (i & (BITS_PER_LONG-1))) )
```

Sets the i^{th} bit of the given bitmap.

Parameters

<i>b</i>	The target bitmap.
<i>i</i>	The bit to be set.

6.13.2.4 UNSET_BIT

```
#define UNSET_BIT(  
    b,  
    i ) ( b[i / BITS_PER_LONG] &= (unsigned long)~(1 << (i & (BITS_PER_LONG-1))) )
```

Unsets the i^{th} bit of the given bitmap.

Parameters

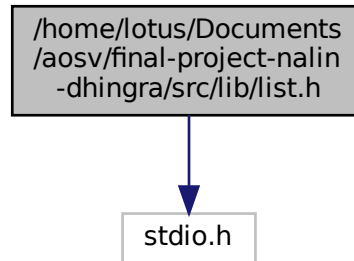
<i>b</i>	The target bitmap.
<i>i</i>	The bit to be unset.

6.14 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/list.h File Reference

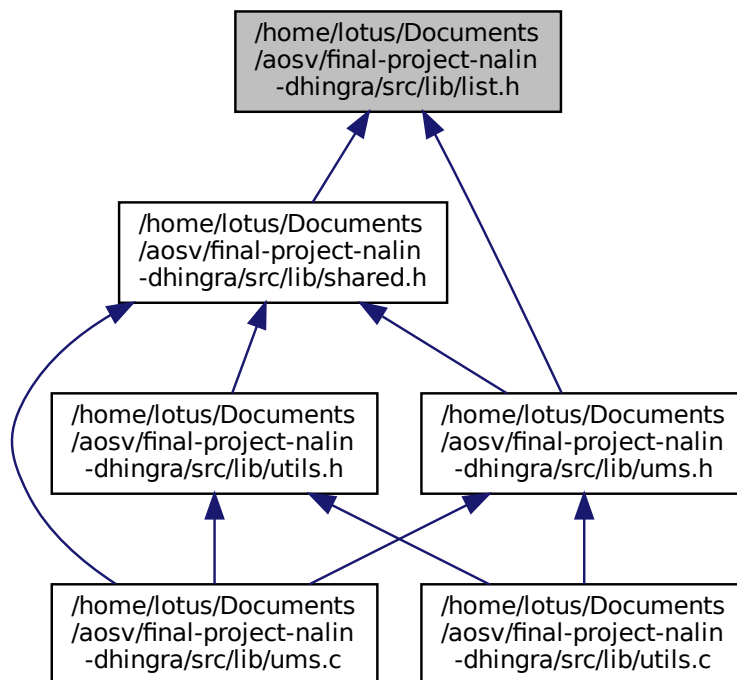
Implementation for linked lists like in the Linux Kernel.

```
#include <stdio.h>
```

Include dependency graph for list.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [list_head](#)
- struct [hlist_head](#)
- struct [hlist_node](#)

Macros

- `#define _GNU_SOURCE`
- `#define typeof __typeof__`
- `#define offsetof(TYPE, MEMBER) ((size_t) &((TYPE *)0)->MEMBER)`
- `#define container_of(ptr, type, member)`
- `#define LIST_POISON1 ((void *) 0x00100100)`
- `#define LIST_POISON2 ((void *) 0x00200200)`
- `#define LIST_HEAD_INIT(name) { &(name), &(name) }`
- `#define LIST_HEAD(name) struct list_head name = LIST_HEAD_INIT(name)`
- `#define list_entry(ptr, type, member) container_of(ptr, type, member)`
- `#define list_first_entry(ptr, type, member) list_entry((ptr)->next, type, member)`
- `#define list_for_each(pos, head) for (pos = (head)->next; pos != (head); pos = pos->next)`
- `#define __list_for_each(pos, head) for (pos = (head)->next; pos != (head); pos = pos->next)`
- `#define list_for_each_prev(pos, head) for (pos = (head)->prev; pos != (head); pos = pos->prev)`
- `#define list_for_each_safe(pos, n, head)`
- `#define list_for_each_prev_safe(pos, n, head)`
- `#define list_for_each_entry(pos, head, member)`
- `#define list_for_each_entry_reverse(pos, head, member)`
- `#define list_prepare_entry(pos, head, member) ((pos) ? : list_entry(head, typeof(*pos), member))`
- `#define list_for_each_entry_continue(pos, head, member)`
- `#define list_for_each_entry_continue_reverse(pos, head, member)`
- `#define list_for_each_entry_from(pos, head, member)`
- `#define list_for_each_entry_safe(pos, n, head, member)`
- `#define list_for_each_entry_safe_continue(pos, n, head, member)`
- `#define list_for_each_entry_safe_from(pos, n, head, member)`
- `#define list_for_each_entry_safe_reverse(pos, n, head, member)`
- `#define list_safe_reset_next(pos, n, member) n = list_entry(pos->member.next, typeof(*pos), member)`
- `#define HLIST_HEAD_INIT { .first = NULL }`
- `#define HLIST_HEAD(name) struct hlist_head name = { .first = NULL }`
- `#define INIT_HLIST_HEAD(ptr) ((ptr)->first = NULL)`
- `#define hlist_entry(ptr, type, member) container_of(ptr, type, member)`
- `#define hlist_for_each(pos, head) for (pos = (head)->first; pos ; pos = pos->next)`
- `#define hlist_for_each_safe(pos, n, head)`
- `#define hlist_for_each_entry(tpos, pos, head, member)`
- `#define hlist_for_each_entry_continue(tpos, pos, member)`
- `#define hlist_for_each_entry_from(tpos, pos, member)`
- `#define hlist_for_each_entry_safe(tpos, pos, n, head, member)`

6.14.1 Detailed Description

Implementation for linked lists like in the Linux Kernel.

This file contains and implementation of the linked lists used in the Linux Kernel replicated to work in userspace.

Bug No known bugs.

6.14.2 Macro Definition Documentation

6.14.2.1 __list_for_each

```
#define __list_for_each(  
    pos,  
    head ) for (pos = (head)->next; pos != (head); pos = pos->next)
```

`__list_for_each` - iterate over a list

Parameters

<i>pos</i>	the &struct list_head to use as a loop cursor.
<i>head</i>	the head for your list.

This variant doesn't differ from [list_for_each\(\)](#) any more. We don't do prefetching in either case.

6.14.2.2 container_of

```
#define container_of(
    ptr,
    type,
    member )
```

Value:

```
((
    const typeof( ((type *)0)->member ) *__mptr = (ptr); \
    (type *) ( (char *)__mptr - offsetof(type,member) ); ))
```

container_of - cast a member of a structure out to the containing structure

Parameters

<i>ptr</i>	the pointer to the member.
<i>type</i>	the type of the container struct this is embedded in.
<i>member</i>	the name of the member within the struct.

6.14.2.3 hlist_for_each_entry

```
#define hlist_for_each_entry(
    tpos,
    pos,
    head,
    member )
```

Value:

```
for (pos = (head)->first; \
    pos && \
    ({ tpos = hlist_entry(pos, typeof(*tpos), member); 1; }); \
    pos = pos->next)
```

hlist_for_each_entry - iterate over list of given type

Parameters

<i>tpos</i>	the type * to use as a loop cursor.
<i>pos</i>	the &struct hlist_node to use as a loop cursor.
<i>head</i>	the head for your list.
<i>member</i>	the name of the hlist_node within the struct.

6.14.2.4 hlist_for_each_entry_continue

```
#define hlist_for_each_entry_continue(
    tpos,
    pos,
    member )
```

Value:

```
for (pos = (pos)->next;
    pos &&
    ({ tpos = hlist_entry(pos, typeof(*tpos), member); 1;}); \
    pos = pos->next)
```

`hlist_for_each_entry_continue` - iterate over a hlist continuing after current point

Parameters

<i>tpos</i>	the type * to use as a loop cursor.
<i>pos</i>	the &struct hlist_node to use as a loop cursor.
<i>member</i>	the name of the hlist_node within the struct.

6.14.2.5 hlist_for_each_entry_from

```
#define hlist_for_each_entry_from(
    tpos,
    pos,
    member )
```

Value:

```
for (; pos &&
    ({ tpos = hlist_entry(pos, typeof(*tpos), member); 1;}); \
    pos = pos->next)
```

`hlist_for_each_entry_from` - iterate over a hlist continuing from current point

Parameters

<i>tpos</i>	the type * to use as a loop cursor.
<i>pos</i>	the &struct hlist_node to use as a loop cursor.
<i>member</i>	the name of the hlist_node within the struct.

6.14.2.6 hlist_for_each_entry_safe

```
#define hlist_for_each_entry_safe(
    tpos,
```

```

    pos,
    n,
    head,
    member )

```

Value:

```

for (pos = (head)->first;
    pos && ({ n = pos->next; 1; }) &&
    ({ tpos = hlist_entry(pos, typeof(*tpos), member); 1;}); \
    pos = n)

```

`hlist_for_each_entry_safe` - iterate over list of given type safe against removal of list entry

Parameters

<i>tpos</i>	the type * to use as a loop cursor.
<i>pos</i>	the &struct hlist_node to use as a loop cursor.
<i>n</i>	another &struct hlist_node to use as temporary storage
<i>head</i>	the head for your list.
<i>member</i>	the name of the hlist_node within the struct.

6.14.2.7 hlist_for_each_safe

```

#define hlist_for_each_safe(
    pos,
    n,
    head )

```

Value:

```

for (pos = (head)->first; pos && ({ n = pos->next; 1; }); \
    pos = n)

```

6.14.2.8 list_entry

```

#define list_entry(
    ptr,
    type,
    member ) container_of(ptr, type, member)

```

`list_entry` - get the struct for this entry

Parameters

<i>ptr</i>	the &struct list_head pointer.
<i>type</i>	the type of the struct this is embedded in.
<i>member</i>	the name of the list_struct within the struct.

6.14.2.9 list_first_entry

```
#define list_first_entry(  
    ptr,  
    type,  
    member ) list_entry((ptr)->next, type, member)
```

list_first_entry - get the first element from a list

Parameters

<i>ptr</i>	the list head to take the element from.
<i>type</i>	the type of the struct this is embedded in.
<i>member</i>	the name of the list_struct within the struct.

Note, that list is expected to be not empty.

6.14.2.10 list_for_each

```
#define list_for_each(  
    pos,  
    head ) for (pos = (head)->next; pos != (head); pos = pos->next)
```

list_for_each - iterate over a list

Parameters

<i>pos</i>	the &struct list_head to use as a loop cursor.
<i>head</i>	the head for your list.

6.14.2.11 list_for_each_entry

```
#define list_for_each_entry(  
    pos,  
    head,  
    member )
```

Value:

```
for (pos = list_entry((head)->next, typeof(*pos), member);  
    &pos->member != (head);  
    pos = list_entry(pos->member.next, typeof(*pos), member))
```

list_for_each_entry - iterate over list of given type

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

6.14.2.12 list_for_each_entry_continue

```
#define list_for_each_entry_continue(  
    pos,  
    head,  
    member )
```

Value:

```
for (pos = list_entry(pos->member.next, typeof(*pos), member);  
    &pos->member != (head);  
    pos = list_entry(pos->member.next, typeof(*pos), member))
```

list_for_each_entry_continue - continue iteration over list of given type

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

Continue to iterate over list of given type, continuing after the current position.

6.14.2.13 list_for_each_entry_continue_reverse

```
#define list_for_each_entry_continue_reverse(  
    pos,  
    head,  
    member )
```

Value:

```
for (pos = list_entry(pos->member.prev, typeof(*pos), member);  
    &pos->member != (head);  
    pos = list_entry(pos->member.prev, typeof(*pos), member))
```

list_for_each_entry_continue_reverse - iterate backwards from the given point

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

Start to iterate over list of given type backwards, continuing after the current position.

6.14.2.14 list_for_each_entry_from

```
#define list_for_each_entry_from(  
    pos,
```

```

    head,
    member )

```

Value:

```

for (; &pos->member != (head); \
    pos = list_entry(pos->member.next, typeof(*pos), member))

```

`list_for_each_entry_from` - iterate over list of given type from the current point

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

Iterate over list of given type, continuing from current position.

6.14.2.15 list_for_each_entry_reverse

```

#define list_for_each_entry_reverse(
    pos,
    head,
    member )

```

Value:

```

for (pos = list_entry((head)->prev, typeof(*pos), member); \
    &pos->member != (head); \
    pos = list_entry(pos->member.prev, typeof(*pos), member))

```

`list_for_each_entry_reverse` - iterate backwards over list of given type.

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

6.14.2.16 list_for_each_entry_safe

```

#define list_for_each_entry_safe(
    pos,
    n,
    head,
    member )

```

Value:

```

for (pos = list_entry((head)->next, typeof(*pos), member), \
    n = list_entry(pos->member.next, typeof(*pos), member); \
    &pos->member != (head); \
    pos = n, n = list_entry(n->member.next, typeof(*n), member))

```

`list_for_each_entry_safe` - iterate over list of given type safe against removal of list entry

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>n</i>	another type * to use as temporary storage
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

6.14.2.17 list_for_each_entry_safe_continue

```
#define list_for_each_entry_safe_continue(
    pos,
    n,
    head,
    member )
```

Value:

```
for (pos = list_entry(pos->member.next, typeof(*pos), member),
     n = list_entry(pos->member.next, typeof(*pos), member);
     &pos->member != (head);
     pos = n, n = list_entry(n->member.next, typeof(*n), member))
```

list_for_each_entry_safe_continue - continue list iteration safe against removal

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>n</i>	another type * to use as temporary storage
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

Iterate over list of given type, continuing after current point, safe against removal of list entry.

6.14.2.18 list_for_each_entry_safe_from

```
#define list_for_each_entry_safe_from(
    pos,
    n,
    head,
    member )
```

Value:

```
for (n = list_entry(pos->member.next, typeof(*pos), member);
     &pos->member != (head);
     pos = n, n = list_entry(n->member.next, typeof(*n), member))
```

list_for_each_entry_safe_from - iterate over list from current point safe against removal

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>n</i>	another type * to use as temporary storage
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

Iterate over list of given type from current point, safe against removal of list entry.

6.14.2.19 list_for_each_entry_safe_reverse

```
#define list_for_each_entry_safe_reverse(
    pos,
    n,
    head,
    member )
```

Value:

```
for (pos = list_entry((head)->prev, typeof(*pos), member), \
     n = list_entry(pos->member.prev, typeof(*pos), member); \
     &pos->member != (head); \
     pos = n, n = list_entry(n->member.prev, typeof(*n), member))
```

list_for_each_entry_safe_reverse - iterate backwards over list safe against removal

Parameters

<i>pos</i>	the type * to use as a loop cursor.
<i>n</i>	another type * to use as temporary storage
<i>head</i>	the head for your list.
<i>member</i>	the name of the list_struct within the struct.

Iterate backwards over list of given type, safe against removal of list entry.

6.14.2.20 list_for_each_prev

```
#define list_for_each_prev(
    pos,
    head ) for (pos = (head)->prev; pos != (head); pos = pos->prev)
```

list_for_each_prev - iterate over a list backwards

Parameters

<i>pos</i>	the &struct list_head to use as a loop cursor.
<i>head</i>	the head for your list.

6.14.2.21 list_for_each_prev_safe

```
#define list_for_each_prev_safe(
    pos,
    n,
    head )
```

Value:

```
for (pos = (head)->prev, n = pos->prev; \
     pos != (head); \
     pos = n, n = pos->prev)
```

`list_for_each_prev_safe` - iterate over a list backwards safe against removal of list entry

Parameters

<i>pos</i>	the &struct list_head to use as a loop cursor.
<i>n</i>	another &struct list_head to use as temporary storage
<i>head</i>	the head for your list.

6.14.2.22 `list_for_each_safe`

```
#define list_for_each_safe(
    pos,
    n,
    head )
```

Value:

```
for (pos = (head)->next, n = pos->next; pos != (head); \
     pos = n, n = pos->next)
```

`list_for_each_safe` - iterate over a list safe against removal of list entry

Parameters

<i>pos</i>	the &struct list_head to use as a loop cursor.
<i>n</i>	another &struct list_head to use as temporary storage
<i>head</i>	the head for your list.

6.14.2.23 `list_prepare_entry`

```
#define list_prepare_entry(
    pos,
    head,
    member ) ((pos) ? : list\_entry(head, typeof(*pos), member))
```

`list_prepare_entry` - prepare a pos entry for use in [list_for_each_entry_continue\(\)](#)

Parameters

<i>pos</i>	the type * to use as a start point
<i>head</i>	the head of the list
<i>member</i>	the name of the list_struct within the struct.

Prepares a pos entry for use as a start point in [list_for_each_entry_continue\(\)](#).

6.14.2.24 list_safe_reset_next

```
#define list_safe_reset_next(  
    pos,  
    n,  
    member ) n = list_entry(pos->member.next, typeof(*pos), member)
```

list_safe_reset_next - reset a stale list_for_each_entry_safe loop

Parameters

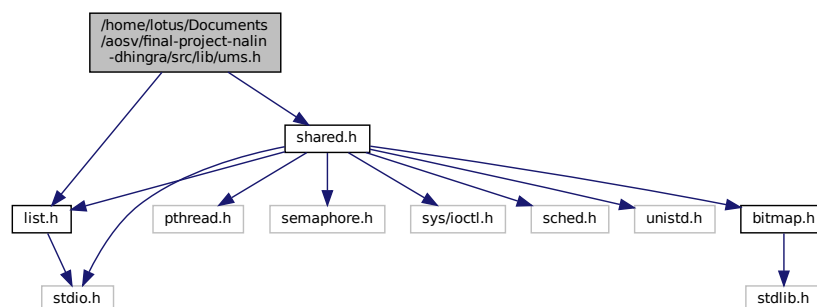
<i>pos</i>	the loop cursor used in the list_for_each_entry_safe loop
<i>n</i>	temporary storage used in list_for_each_entry_safe
<i>member</i>	the name of the list_struct within the struct.

list_safe_reset_next is not safe to use in general if the list may be modified concurrently (eg. the lock is dropped in the loop body). An exception to this is if the cursor element (pos) is pinned in the list, and list_safe_reset_next is called after re-taking the lock and before completing the current iteration of the loop body.

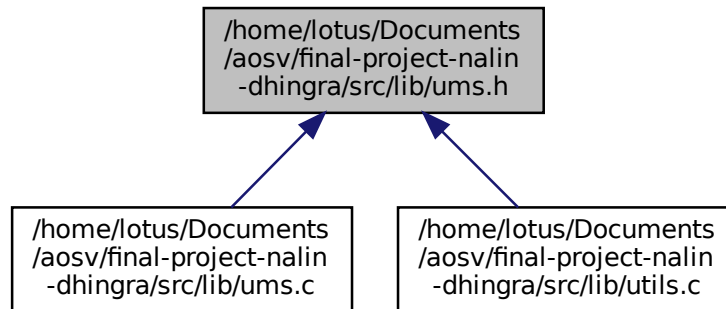
6.15 /home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h File Reference

Header file exposing the main user API to interact with the UMS driver.

```
#include "list.h"  
#include "shared.h"  
Include dependency graph for ums.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- int [ums_init](#) (void)
Initializes the UMS driver for the current process.
- void [ums_destroy](#) (void)
Cleans up all UMS data created by a process.
- int [ums_worker_create](#) (struct [ums_worker](#) *worker, void(*start_routine)(void *), void *arg)
Creates a UMS worker.
- int [ums_worker_list_insert](#) (struct [list_head](#) *head, struct [ums_worker](#) *worker)
Inserts a UMS worker into a completion list.
- int [EnterUmsSchedulingMode](#) (void(*scheduler_routine)(), struct [list_head](#) *worker_list)
Initializes the scheduler thread with its relative worker list.
- int [ExecuteUmsThread](#) (struct [ums_worker](#) *worker)
Executes the given worker in the scheduler's context.
- int [UmsThreadYield](#) (void)
Yields the current worker and restores the scheduler's context.
- struct [list_head](#) * [DequeueUmsCompletionListItems](#) (size_t nworkers)
Returns a list of valid workers that are available to be executed.

6.15.1 Detailed Description

Header file exposing the main user API to interact with the UMS driver.

This file contains the main data structure and variables needed by the UMS library to interact with the IOCTL driver. In particular it implements the main endpoints used by the user applications.

Author

Nalin Dhingra (Lotus98)

Bug No known bugs.

6.15.2 Function Documentation

6.15.2.1 DequeueUmsCompletionListItems()

```
struct list_head* DequeueUmsCompletionListItems (
    size_t nworkers )
```

Returns a list of valid workers that are available to be executed.

This function is a blocking function and will return a sublist of workers from the scheduler's completion list, that are available to be executed on this scheduler.

Parameters

<i>nworkers</i>	The number of workers wanted in the queue. If the value is either equal to 0 or greater then the number available workers, the latter value will be used instead.
-----------------	---

Returns

struct [list_head](#) *: A pointer to a list head that will contain valid workers. The list has to be freed by the user.

A pointer to a struct [list_head](#) representing the dequeued list or NULL if there are no more workers to be executed.

6.15.2.2 EnterUmsSchedulingMode()

```
int EnterUmsSchedulingMode (
    void(*)() scheduler_routine,
    struct list_head * worker_list )
```

Initializes the scheduler thread with its relative worker list.

This function creates a scheduler thread and assigns it to the first non used CPU for this process's UMS. If there are no more CPUs available to the process it will return an error. It also assigns the given worker_list to that scheduler, performing additional operations to register both components to the UMS driver. This function must only be called by the main thread, any other usage, may cause unwanted behaviour.

Parameters

<i>scheduler_routine</i>	The scheduling function that will be executed by the scheduler thread.
<i>worker_list</i>	The worker list to be assigned to the scheduler.

Returns

0 if successful <0 if failed.

6.15.2.3 ExecuteUmsThread()

```
int ExecuteUmsThread (
    struct ums_worker * worker )
```

Executes the given worker in the scheduler's context.

Parameters

<i>worker</i>	The worker to be executed.
---------------	----------------------------

Returns

0 if successful <0 if failed.

6.15.2.4 ums_destroy()

```
void ums_destroy (
    void )
```

Cleans up all UMS data created by a process.

This function is to be called when a process is terminating its UMS session. Calling this function when a thread is still running may result in unexpected behaviour.

6.15.2.5 ums_init()

```
int ums_init (
    void )
```

Initializes the UMS driver for the current process.

This function is to be called by the main thread before starting a UMS session.

Returns

0 if successful <0 if failed.

6.15.2.6 ums_worker_create()

```
int ums_worker_create (
    struct ums_worker * worker,
    void(*) (void *) start_routine,
    void * arg )
```

Creates a UMS worker.

This function is used as a wrapper for `pthread_create`, to instantiate a UMS thread.

Parameters

<i>worker</i>	A pointer to a struct ums_worker that will be initialized.
<i>start_routine</i>	The routine that will be executed by the UMS thread.
<i>arg</i>	The argument needed by the UMS thread routine.

Returns

0 if successful <0 if failed.

6.15.2.7 ums_worker_list_insert()

```
int ums_worker_list_insert (
    struct list\_head * head,
    struct ums\_worker * worker )
```

Inserts a UMS worker into a completion list.

Parameters

<i>head</i>	The head of the list.
<i>worker</i>	The worker to be inserted as a node.

Returns

0 if successful <0 if failed.

Index

/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
21	bitmap.h, 58
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
22	ioctl.h, 24
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
25	utils.c, 41
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
27	DequeueUmsCompletionListItems
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
29	ums.h, 73
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
35	utils.c, 45
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
39	utils.h, 54
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/dev/queue_list	DECLARE_BITMAP
47	ums.c, 37
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
56	EXECUTE_THREAD
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
59	execute_thread
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
32	utils.h, 50
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
35	ums.c, 37
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
71	ums.h, 73
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
44	utils.c, 45
/home/lotus/Documents/aosv/final-project-nalin-dhingra/src/lib/ums.h	DECLARE_BITMAP
52	utils.h, 54
__list_for_each	find_ums_proc
list.h, 61	utils.c, 42
	utils.h, 50
bitmap.h	find_worker_tid
DECLARE_BITMAP, 58	utils.c, 46
GET_BIT, 58	utils.h, 55
SET_BIT, 58	GET_BIT
UNSET_BIT, 59	bitmap.h, 58
cleanup_process	hlist_for_each_entry
utils.c, 41	list.h, 62
utils.h, 49	hlist_for_each_entry_continue
container_of	list.h, 63
list.h, 62	hlist_for_each_entry_from
cpuid	list.h, 63
ums_sched_arg, 15	hlist_for_each_entry_safe
ums_usr_worker, 16	list.h, 63
current_worker	hlist_for_each_safe
ums sched, 14	list.h, 64
	hlist head, 9

- hlist_node, [10](#)
- INIT_WORKER
 - ioctl.h, [24](#)
- init_worker_node
 - utils.c, [42](#)
 - utils.h, [50](#)
- ioctl.c
 - ums_ioctl, [22](#)
- ioctl.h
 - DEQUEUE_LIST, [24](#)
 - EXECUTE_THREAD, [24](#)
 - INIT_WORKER, [24](#)
 - REGISTER_SCHED, [24](#)
 - REGISTER_WORKER, [25](#)
 - ums_ioctl, [25](#)
- list.h
 - __list_for_each, [61](#)
 - container_of, [62](#)
 - hlist_for_each_entry, [62](#)
 - hlist_for_each_entry_continue, [63](#)
 - hlist_for_each_entry_from, [63](#)
 - hlist_for_each_entry_safe, [63](#)
 - hlist_for_each_safe, [64](#)
 - list_entry, [64](#)
 - list_first_entry, [64](#)
 - list_for_each, [65](#)
 - list_for_each_entry, [65](#)
 - list_for_each_entry_continue, [66](#)
 - list_for_each_entry_continue_reverse, [66](#)
 - list_for_each_entry_from, [66](#)
 - list_for_each_entry_reverse, [67](#)
 - list_for_each_entry_safe, [67](#)
 - list_for_each_entry_safe_continue, [68](#)
 - list_for_each_entry_safe_from, [68](#)
 - list_for_each_entry_safe_reverse, [69](#)
 - list_for_each_prev, [69](#)
 - list_for_each_prev_safe, [69](#)
 - list_for_each_safe, [70](#)
 - list_prepare_entry, [70](#)
 - list_safe_reset_next, [70](#)
- list_entry
 - list.h, [64](#)
- list_first_entry
 - list.h, [64](#)
- list_for_each
 - list.h, [65](#)
- list_for_each_entry
 - list.h, [65](#)
- list_for_each_entry_continue
 - list.h, [66](#)
- list_for_each_entry_continue_reverse
 - list.h, [66](#)
- list_for_each_entry_from
 - list.h, [66](#)
- list_for_each_entry_reverse
 - list.h, [67](#)
- list_for_each_entry_safe
 - list.h, [67](#)
- list_for_each_entry_safe_continue
 - list.h, [68](#)
- list_for_each_entry_safe_from
 - list.h, [68](#)
- list_for_each_entry_safe_reverse
 - list.h, [69](#)
- list_for_each_prev
 - list.h, [69](#)
- list_for_each_prev_safe
 - list.h, [69](#)
- list_for_each_safe
 - list.h, [70](#)
- list_head, [10](#)
- list_prepare_entry
 - list.h, [70](#)
- list_safe_reset_next
 - list.h, [70](#)
- ncpus
 - shared.h, [31](#), [34](#)
- open_device
 - utils.c, [46](#)
 - utils.h, [55](#)
- procfs.c
 - sched_proc_read, [27](#)
 - worker_proc_read, [27](#)
- procfs.h
 - sched_proc_read, [28](#)
 - worker_proc_read, [29](#)
- procfs_proc_umsdata, [11](#)
- procfs_sched_umsdata, [11](#)
- procfs_worker_umsdata, [12](#)
- REGISTER_SCHED
 - ioctl.h, [24](#)
- register_ums_process
 - utils.c, [42](#)
 - utils.h, [51](#)
- register_ums_scheduler
 - utils.c, [43](#)
 - utils.h, [51](#)
- register_usr_worker
 - utils.c, [43](#)
 - utils.h, [51](#)
- REGISTER_WORKER
 - ioctl.h, [25](#)
- sched_proc_read
 - procfs.c, [27](#)
 - procfs.h, [28](#)
- sched_wrap_routine
 - utils.c, [46](#)
 - utils.h, [55](#)
- SET_BIT
 - bitmap.h, [58](#)
- shared.h

- ncpus, [31](#), [34](#)
- state
 - ums_worker, [18](#)
- terminate_ums_process
 - utils.c, [44](#)
 - utils.h, [52](#)
- tid
 - ums_usr_worker, [17](#)
- ums.c
 - DequeueUmsCompletionListItems, [37](#)
 - EnterUmsSchedulingMode, [37](#)
 - ExecuteUmsThread, [37](#)
 - ums_destroy, [38](#)
 - ums_init, [38](#)
 - ums_worker_create, [38](#)
 - ums_worker_list_insert, [39](#)
- ums.h
 - DequeueUmsCompletionListItems, [73](#)
 - EnterUmsSchedulingMode, [73](#)
 - ExecuteUmsThread, [73](#)
 - ums_destroy, [74](#)
 - ums_init, [74](#)
 - ums_worker_create, [74](#)
 - ums_worker_list_insert, [75](#)
- ums_destroy
 - ums.c, [38](#)
 - ums.h, [74](#)
- ums_init
 - ums.c, [38](#)
 - ums.h, [74](#)
- ums_ioctl
 - ioctl.c, [22](#)
 - ioctl.h, [25](#)
- ums_proc, [12](#)
- ums_sched, [13](#)
 - current_worker, [14](#)
- ums_sched_arg, [15](#)
 - cpuid, [15](#)
- ums_thread, [16](#)
- ums_usr_worker, [16](#)
 - cpuid, [16](#)
 - tid, [17](#)
- ums_worker, [17](#)
 - state, [18](#)
- ums_worker_arg, [19](#)
- ums_worker_create
 - ums.c, [38](#)
 - ums.h, [74](#)
- ums_worker_list_insert
 - ums.c, [39](#)
 - ums.h, [75](#)
- ums_worker_node, [20](#)
- UNSET_BIT
 - bitmap.h, [59](#)
- utils.c
 - cleanup_process, [41](#)
 - dequeue_list, [41](#)
 - dup_worker_list, [45](#)
 - execute_thread, [41](#)
 - find_next_zero_bit, [45](#)
 - find_ums_proc, [42](#)
 - find_worker_tid, [46](#)
 - init_worker_node, [42](#)
 - open_device, [46](#)
 - register_ums_process, [42](#)
 - register_ums_scheduler, [43](#)
 - register_usr_worker, [43](#)
 - sched_wrap_routine, [46](#)
 - terminate_ums_process, [44](#)
 - worker_wrap_routine, [47](#)
- utils.h
 - cleanup_process, [49](#)
 - dequeue_list, [49](#)
 - dup_worker_list, [54](#)
 - execute_thread, [50](#)
 - find_next_zero_bit, [54](#)
 - find_ums_proc, [50](#)
 - find_worker_tid, [55](#)
 - init_worker_node, [50](#)
 - open_device, [55](#)
 - register_ums_process, [51](#)
 - register_ums_scheduler, [51](#)
 - register_usr_worker, [51](#)
 - sched_wrap_routine, [55](#)
 - terminate_ums_process, [52](#)
 - worker_wrap_routine, [56](#)
- worker_proc_read
 - procfs.c, [27](#)
 - procfs.h, [29](#)
- worker_wrap_routine
 - utils.c, [47](#)
 - utils.h, [56](#)